JavaScript

do zero ao avançado

Guia completo em português: linguagem, navegador, Node.js, ferramentas e as principais bibliotecas.

```
const futuro = await aprender();
```

Edição de novembro de 2026

Sumário

Antes de começar 4

Como usar este guia . . . 5

Fundamentos 7

1. Primeiros passos . . . 8
2. Variáveis e tipos de dados . . . 10
3. Operadores . . . 12
4. Condições . . . 14
5. Repetições (loops) . . . 16
6. Funções . . . 18
7. Arrays (listas) . . . 21
8. Objetos e dicionários . . . 24
9. Strings, números e datas . . . 27

Linguagem avançada 29

10. Erros e depuração . . . 30
11. Módulos . . . 32
12. Classes e programação orientada a objetos . . . 34
13. Código assíncrono . . . 36
14. DOM e eventos (JavaScript no navegador) . . . 39
15. Boas práticas e ferramentas . . . 42

Ecossistema e bibliotecas 44

16. Node.js e npm . . . 45
17. Entendendo bibliotecas . . . 48
18. Bibliotecas utilitárias . . . 50
19. Bibliotecas e frameworks de front-end . . . 53
20. Back-end com Node.js . . . 57
21. Testes automatizados . . . 61
22. TypeScript . . . 63

Prática e referência 65

23. Projetos para praticar . . . 66

24. Roteiro de estudos de 16 semanas 68
Apêndice A: Gabarito de exercícios selecionados 70
Apêndice B: Cola rápida de sintaxe 74
Apêndice C: Glossário 77
Apêndice D: Onde continuar aprendendo 78

INÍCIO

Antes de começar

Como usar este guia, o que você vai aprender e como organizar seus estudos para não travar no meio do caminho.

Como usar este guia

Para quem é este material

Este guia leva você do primeiro `console.log` até projetos reais com Node.js e bibliotecas populares. Não é preciso saber programar antes. Se você já conhece outra linguagem, pode ler os capítulos iniciais mais rápido e focar nas diferenças do JavaScript.

O conteúdo está dividido em quatro partes:

Parte	O que você aprende	Capítulos
Fundamentos	Variáveis, tipos, condições, loops, funções, arrays, objetos, strings, números e datas	1 a 9
Linguagem avançada	Erros, módulos, classes, assincronismo, DOM, boas práticas	10 a 15
Ecossistema e bibliotecas	Node.js, npm, bibliotecas utilitárias, front-end, back-end, testes, TypeScript	16 a 22
Prática	Projetos, roteiro de estudos, gabaritos, referências	23 a 24 e apêndices

Como estudar com este guia

A programação se aprende fazendo, não lendo. Para cada capítulo, siga este ciclo:

1. Leia a seção e **digite** os exemplos (não copie e cole). Digitar fixa a sintaxe.
2. Altere os exemplos: mude valores, quebre de propósito, veja a mensagem de erro.
3. Resolva os exercícios do final do capítulo antes de olhar o gabarito do Apêndice A.
4. Só avance quando conseguir explicar o assunto com suas palavras.

> **Dica:** estude pouco todos os dias. Quarenta e cinco minutos por dia rendem mais do que um dia inteiro por semana. O roteiro do capítulo 24 já vem nesse formato.

O que você precisa

- Um computador com navegador moderno (Chrome, Firefox, Edge ou Safari).
- Um editor de código. Recomendamos o Visual Studio Code (gratuito).
- O Node.js, a partir do capítulo 1 (também gratuito).
- Conexão com a internet para consultar a documentação e instalar bibliotecas.

Convenções usadas

Trechos de código aparecem em caixas com fundo cinza. Comentários explicam o resultado esperado:

```
console.log(2 + 3); // 5
```

Caixas coloridas destacam avisos e dicas:

> **Dica:** sugestões que aceleram seu aprendizado.

> **Atenção:** armadilhas comuns que causam bugs difíceis de achar.

Nota sobre versões e bibliotecas

A linguagem JavaScript muda devagar e mantém compatibilidade, então o que você aprende nas partes I e II vale por muitos anos. Já as bibliotecas e ferramentas da parte III evoluem rápido: nomes de comandos, opções e até as bibliotecas mais populares mudam com o tempo. Por isso, o guia ensina os **conceitos** e mostra exemplos curtos; para detalhes atuais, consulte sempre a documentação oficial de cada projeto (links no Apêndice D).

PARTE I

Fundamentos

Tudo o que você precisa para ler e escrever programas simples: dados, decisões, repetições e funções.

1. Primeiros passos

O que é JavaScript

JavaScript (JS) é a linguagem de programação da web. Ela roda dentro dos navegadores, onde controla páginas, botões, formulários e animações, e também fora deles, graças ao **Node.js**, que permite criar servidores, ferramentas de linha de comando e automações. Com a mesma linguagem você faz sites, aplicativos web, APIs, apps de celular (React Native) e programas de desktop (Electron).

> **Nota:** apesar do nome parecido, JavaScript e Java são linguagens diferentes, com usos e regras diferentes.

A linguagem segue um padrão chamado **ECMAScript**, atualizado todo ano. A versão de 2015 (ES2015, ou ES6) trouxe `let`, `const`, funções de seta, classes, módulos e promessas. Este guia usa o JavaScript moderno.

Onde executar seu código

Você tem três formas de rodar JavaScript, e vai usar todas:

Local	Como usar	Bom para
Console do navegador	Tecla F12, aba Console	Testar uma linha rapidamente
Página HTML	Tag script dentro de um arquivo .html	Sites e interação com a página
Node.js	Comando node arquivo.js no terminal	Servidores, scripts, ferramentas

Para instalar o Node.js, baixe a versão **LTS** (suporte de longo prazo) em nodejs.org. Depois confirme no terminal:

```
node --version
npm --version
```

Seu primeiro programa

Crie um arquivo chamado `ola.js` com este conteúdo:

```
console.log("Olá, mundo!");
```

Execute no terminal com `node ola.js`. A saída será `Olá, mundo!`.

Para rodar no navegador, crie `index.html` e `app.js` na mesma pasta:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Meu primeiro site</title>
</head>
<body>
  <h1>Olá!</h1>
  <script src="app.js"></script>
</body>
</html>
```

Coloque `console.log("Funcionou!");` dentro de `app.js`, abra o HTML no navegador e veja a mensagem no Console (F12).

Comentários e o console

Comentários são ignorados pelo computador e servem para explicar o código:

```
// comentário de uma linha
/* comentário
   de várias linhas */
console.log("texto normal");
console.warn("aviso");
console.error("erro");
console.table([{ nome: "Ana", idade: 25 }, { nome: "Bia", idade: 30 }]);
```

O ponto e vírgula no fim das linhas é opcional em JavaScript, mas escolha um estilo e mantenha em todo o projeto. Neste guia usamos ponto e vírgula.

Exercícios

1. Instale o Node.js e execute `node --version`.
2. Crie um arquivo `ola.js` que mostre seu nome e sua cidade em duas linhas.
3. Crie uma página HTML com um `<h1>` e um script que mostre uma mensagem no Console.
4. Use `console.table` para mostrar uma lista de três pessoas com nome e idade.

2. Variáveis e tipos de dados

Declarando variáveis

Uma variável é um nome que guarda um valor. Em JavaScript moderno existem duas formas principais:

```
let idade = 25;       // pode ser reatribuída
const nome = "Ana";   // não pode ser reatribuída
idade = 26;           // ok
// nome = "Bia";      // ERRO: Assignment to constant variable
```

Regra prática: use `const` por padrão e troque para `let` apenas quando precisar alterar o valor. Existe também o `var`, que é a forma antiga e tem regras de escopo confusas. Evite-o em código novo.

Nomes de variáveis usam **camelCase** (`precoFinal`, `nomeCompleto`), não começam com número e não podem ser palavras reservadas como `class` ou `return`.

Os tipos de dados

Tipo	Exemplo	Descrição
string	"texto" ou 'texto'	Texto (aspas simples, duplas ou crases)
number	42, 3.14, -7	Inteiros e decimais (um só tipo)
boolean	true, false	Verdadeiro ou falso
undefined	undefined	Variável sem valor atribuído
null	null	Ausência intencional de valor
bigint	123n	Inteiros muito grandes
symbol	Symbol("id")	Identificador único (uso avançado)
object	{}, [], new Date()	Coleções e estruturas

O operador `typeof` informa o tipo:

```
console.log(typeof "oi");       // "string"
console.log(typeof 42);         // "number"
console.log(typeof true);       // "boolean"
console.log(typeof undefined);  // "undefined"
console.log(typeof {});         // "object"
console.log(typeof [1, 2]);     // "object" (arrays são objetos)
console.log(typeof null);       // "object" (erro histórico da linguagem)
```

Strings e template literals

Strings aceitam aspas simples, duplas ou crases. As crases criam **template literals**, que permitem inserir valores com `${}` e escrever em várias linhas:

```
const nome = "Ana";
const idade = 25;
const msg = `Olá, ${nome}! Ano que vem você terá ${idade + 1} anos.`;
console.log(msg);
// Olá, Ana! Ano que vem você terá 26 anos.
```

Números

Inteiros e decimais são o mesmo tipo. Há valores especiais: `Infinity` e `NaN` (Not a Number, resultado de uma conta impossível).

```
console.log(10 / 3);        // 3.3333333333333335
console.log(10 / 0);        // Infinity
console.log("abc" * 2);     // NaN
console.log(0.1 + 0.2);     // 0.30000000000000004
```

> **Atenção:** a imprecisão de `0.1 + 0.2` existe em quase todas as linguagens, porque computadores guardam decimais em binário. Para dinheiro, trabalhe em centavos (inteiros) ou arredonde com `toFixed(2)` na hora de exibir.

null e undefined

`undefined` significa "ainda não recebeu valor". `null` significa "está vazio de propósito".

```
let a;
console.log(a);          // undefined
let usuario = null;      // ainda não há usuário logado
```

Conversão de tipos

```
Number("42");        // 42
Number("abc");       // NaN
parseInt("42px");    // 42
parseFloat("3.14m"); // 3.14
String(123);         // "123"
Boolean(0);          // false
Boolean("texto");    // true
Number.isNaN(NaN);   // true
```

Valores falsy e truthy

Em condições, alguns valores contam como falso (**falsy**): `false`, `0`, `""` (texto vazio), `null`, `undefined`, `NaN` e `0n`. Todos os outros são verdadeiros (**truthy**), inclusive `"0"`, `[]` e `{}`.

```
if ("") console.log("não aparece");
if ("0") console.log("aparece");   // texto "0" é truthy
if ([]) console.log("aparece");    // array vazio é truthy
```

Exercícios

1. Declare variáveis para nome, idade, altura e se a pessoa é estudante. Mostre o tipo de cada uma com `typeof`.
2. Monte a frase `"Meu nome é X e tenho Y anos"` usando template literal.
3. Converta o texto `"15.5"` para número e some 4.5.
4. Descubra o que `Number("")`, `Number(null)` e `Number(undefined)` retornam.
5. Calcule `0.1 + 0.2` e depois mostre o resultado com 2 casas usando `toFixed(2)`.

3. Operadores

Aritméticos

```
let a = 10, b = 3;
console.log(a + b);   // 13
console.log(a - b);   // 7
console.log(a * b);   // 30
console.log(a / b);   // 3.3333333333333335
console.log(a % b);   // 1  (resto da divisão)
console.log(a ** b);  // 1000  (potência)
```

Atalhos de atribuição e incremento:

```
let x = 5;
x += 2;   // x = x + 2  -> 7
x -= 1;   // 6
x *= 3;   // 18
x++;      // 19  (soma 1)
x--;      // 18  (subtrai 1)
```

Comparação

Operador	Significado	Exemplo
===	Igual (valor e tipo)	5 === 5 é true
!==	Diferente	5 !== "5" é true
> e <	Maior e menor	7 > 3 é true
>= e <=	Maior ou igual, menor ou igual	7 >= 7 é true
==	Igual com conversão (evite)	5 == "5" é true

```
console.log(5 === "5");  // false: tipos diferentes
console.log(5 == "5");   // true: o JS converteu o texto
console.log(0 == false); // true (surpresa!)
```

> **Atenção:** use sempre `===` e `!==`. O `==` converte tipos por baixo dos panos e produz resultados difíceis de prever.

Lógicos

`&&` (E), `||` (OU) e `!` (NÃO) combinam condições:

```
const idade = 20;
const temCarteira = true;
console.log(idade >= 18 && temCarteira); // true
console.log(idade < 18 || temCarteira);  // true
console.log(!temCarteira);               // false
```

Eles também funcionam como **curto-circuito**: `&&` e `||` retornam um dos valores, não apenas `true` ou `false`.

```
const nome = "" || "Visitante";   // "Visitante"
const ok = true && "pode entrar"; // "pode entrar"
```

Operadores modernos úteis

O **coalescência nula** (`??`) usa o valor da direita só quando a esquerda é `null` ou `undefined`. Diferente do `||`, ele respeita `0` e `""`:

```
const pontos = 0;
console.log(pontos || 10);  // 10 (descartou o zero)
console.log(pontos ?? 10);  // 0  (zero é um valor válido)
```

O **encadeamento opcional** (`?.`) evita erro ao acessar algo que pode não existir:

```
const usuario = { nome: "Ana" };
console.log(usuario.endereco?.cidade); // undefined (sem erro)
```

O **operador ternário** é um `if` em uma linha:

```
const idade = 17;
const status = idade >= 18 ? "adulto" : "menor";
console.log(status); // "menor"
```

Exercícios

1. Calcule a média de três notas e mostre o resultado.
2. Verifique se um número é divisível por 3 e por 5 ao mesmo tempo.
3. Use o ternário para mostrar `"par"` ou `"ímpar"` de um número.
4. Dado `const config = { tema: "" }`, use `??` e `||` para definir um tema padrão `"claro"` e compare os resultados.
5. Acesse `pessoa.endereco.rua` com `?.` quando `pessoa = { nome: "Ana" }`.

4. Condições

if, else if e else

```
const nota = 7.5;
if (nota >= 7) {
  console.log("Aprovado");
} else if (nota >= 5) {
  console.log("Recuperação");
} else {
  console.log("Reprovado");
}
```

As condições são avaliadas de cima para baixo e a **primeira** verdadeira vence. Por isso a ordem importa: comece pelas mais específicas.

switch

Útil para comparar uma variável com vários valores exatos:

```
const dia = 3;
let nomeDoDia;
switch (dia) {
  case 1:
    nomeDoDia = "Domingo";
    break;
  case 2:
    nomeDoDia = "Segunda";
    break;
  case 3:
    nomeDoDia = "Terça";
    break;
  default:
    nomeDoDia = "Outro dia";
}
console.log(nomeDoDia); // "Terça"
```

Atenção: sem o `break`, a execução "vaza" para o case seguinte. Esquecer o `break` é um erro clássico.

Retorno antecipado

Em funções, prefira tratar os casos de erro primeiro e sair cedo. Isso evita `if` aninhado e deixa o código mais legível:

```
function dividir(a, b) {
  if (b === 0) {
    return "Não é possível dividir por zero";
  }
  return a / b;
}
```

Exercícios

1. Mostre se um número é positivo, negativo ou zero.
2. Dados três números, mostre o maior deles.

3. Descubra se um ano é bissexto (divisível por 4 e não por 100, exceto se divisível por 400).
4. Use `switch` para converter um número de 1 a 12 no nome do mês.
5. Crie uma calculadora simples: dados dois números e um operador (`+`, `-`, `*`, `/`), mostre o resultado.

5. Repetições (loops)

for

Use quando você sabe quantas vezes repetir:

```
for (let i = 1; i <= 5; i++) {
  console.log("Contagem:", i);
}
```

O `for` tem três partes: inicialização (`let i = 1`), condição (`i <= 5`) e passo (`i++`).

while e do...while

Use quando a repetição depende de uma condição:

```
let n = 1;
while (n <= 3) {
  console.log(n);
  n++;
}
let tentativa = 0;
do {
  tentativa++;
  console.log("Tentativa", tentativa);
} while (tentativa < 3);   // do...while executa pelo menos uma vez
```

> **Atenção:** se a condição nunca ficar falsa, o loop é infinito e trava o programa. Sempre garanta que algo dentro do loop muda a condição.

for...of e for...in

`for...of` percorre os **valores** de listas e textos. `for...in` percorre as **chaves** de um objeto:

```
for (const fruta of ["maçã", "uva", "pera"]) {
  console.log(fruta);
}
for (const letra of "JS") {
  console.log(letra); // "J" depois "S"
}
const pessoa = { nome: "Ana", idade: 25 };
for (const chave in pessoa) {
  console.log(chave, "=", pessoa[chave]);
}
```

break e continue

```
for (let i = 1; i <= 10; i++) {
  if (i === 3) continue;  // pula esta volta
  if (i === 6) break;     // encerra o loop
  console.log(i);         // 1, 2, 4, 5
}
```

Padrões comuns

Acumular (somar de 1 a 100) e contar:

```
let soma = 0;
for (let i = 1; i <= 100; i++) {
  soma += i;
}
console.log(soma); // 5050
let pares = 0;
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) pares++;
}
console.log(pares); // 10
```

Loops aninhados (tabuada):

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    console.log(`${i} x ${j} = ${i * j}`);
  }
}
```

Exercícios

1. Mostre os números de 1 a 20.
2. Mostre apenas os números pares de 1 a 50.
3. Calcule o fatorial de um número (5! = 120).
4. Mostre a tabuada de um número escolhido.
5. FizzBuzz: de 1 a 30, mostre "Fizz" para múltiplos de 3, "Buzz" para múltiplos de 5, "FizzBuzz" para ambos e o número nos demais casos.
6. Conte quantas vogais existem em um texto.

6. Funções

O que são funções

Uma função é um bloco de código com nome que você pode executar quantas vezes quiser. Ela recebe **parâmetros** (entradas), faz um trabalho e pode devolver um resultado com `return`. Funções evitam repetição e dão nome às ideias do seu programa.

```
function saudar(nome) {
  return `Olá, ${nome}!`;
}
console.log(saudar("Ana"));    // Olá, Ana!
console.log(saudar("Carlos")); // Olá, Carlos!
```

Chamar uma função sem `return` devolve `undefined`. Quando o `return` é executado, a função termina imediatamente.

Três formas de escrever funções

```
// 1) Declaração de função (pode ser chamada antes de ser definida)
function somar(a, b) {
  return a + b;
}
// 2) Expressão de função (guardada em uma variável)
const subtrair = function (a, b) {
  return a - b;
};
// 3) Função de seta (arrow function), a mais usada hoje
const multiplicar = (a, b) => a * b;
const dobro = n => n * 2;               // um parâmetro: parênteses opcionais
const ola = () => console.log("Olá!");  // sem parâmetros
```

Na função de seta com uma única expressão, o resultado é retornado automaticamente. Com chaves, você precisa escrever `return`:

```
const quadrado = n => {
  const resultado = n * n;
  return resultado;
};
```

Parâmetros padrão, rest e destructuring

```
// valor padrão
function potencia(base, expoente = 2) {
  return base ** expoente;
}
console.log(potencia(5));    // 25
console.log(potencia(2, 3)); // 8
// rest: recebe qualquer quantidade de argumentos como array
function somarTudo(...numeros) {
  let total = 0;
  for (const n of numeros) total += n;
  return total;
}
console.log(somarTudo(1, 2, 3, 4)); // 10
// destructuring no parâmetro: receba um objeto e pegue só o que precisa
function apresentar({ nome, idade = 18 }) {
```

```
  return `${nome}, ${idade} anos`;
}
console.log(apresentar({ nome: "Ana" })); // Ana, 18 anos
```

Para devolver vários valores, retorne um array ou um objeto:

```
function minMax(lista) {
  return { min: Math.min(...lista), max: Math.max(...lista) };
}
const { min, max } = minMax([4, 9, 1, 7]);
console.log(min, max); // 1 9
```

Escopo

Escopo é a região do código onde uma variável existe.

```
const global = "visível em todo lugar";
function teste() {
  const local = "só dentro da função";
  console.log(global, local); // ok
}
teste();
// console.log(local);  // ERRO: local is not defined
if (true) {
  let dentroDoBloco = 1;
}
// console.log(dentroDoBloco);  // ERRO: let e const respeitam blocos { }
```

Dica: declare variáveis no menor escopo possível. Menos variáveis globais significam menos bugs.

Closures

Uma função "lembra" das variáveis do lugar onde foi criada, mesmo depois que esse lugar terminou de executar. Isso se chama **closure** e permite criar estado privado:

```
function criarContador() {
  let total = 0;
  return () => {
    total++;
    return total;
  };
}
const contador = criarContador();
console.log(contador()); // 1
console.log(contador()); // 2
console.log(contador()); // 3
```

A variável `total` não pode ser alterada de fora. Só a função retornada tem acesso a ela.

Funções como valores: callbacks e funções de ordem superior

Em JavaScript, funções são valores comuns. Você pode guardá-las em variáveis, passá-las como argumento e retorná-las de outras funções. Uma função que recebe outra é chamada de **callback**; quem a recebe é uma função de **ordem superior**:

```
function repetir(vezes, acao) {
  for (let i = 0; i < vezes; i++) {
    acao(i);
  }
}
repetir(3, i => console.log("Volta número", i));
setTimeout(() => console.log("passou 1 segundo"), 1000);
```

Os métodos de array `map`, `filter` e `reduce` (capítulo 7) são exemplos famosos dessa ideia.

Recursão

Uma função pode chamar a si mesma. Toda recursão precisa de um **caso base** que a encerre:

```
function fatorial(n) {
  if (n <= 1) return 1;          // caso base
  return n * fatorial(n - 1);    // chamada recursiva
}
console.log(fatorial(5)); // 120
```

Funções puras

Uma função é **pura** quando, dados os mesmos argumentos, sempre devolve o mesmo resultado e não altera nada fora dela. Funções puras são fáceis de testar e de entender. Prefira-as sempre que possível:

```
// pura
const comDesconto = (preco, pct) => preco * (1 - pct / 100);
// impura: altera uma variável externa
let total = 0;
function adicionar(valor) {
  total += valor;
}
```

Exercícios

1. Crie `dobro(n)` e reescreva como arrow function.
2. Crie `ehPar(n)` que retorna `true` ou `false`.
3. Crie `calcularMedia(...notas)` que aceite qualquer quantidade de notas.
4. Crie `contarVogais(texto)` que retorne quantas vogais o texto tem.
5. Crie `criarSaudacao(saudacao)` que retorne uma função: `criarSaudacao("Bom dia")("Ana")` retorna `"Bom dia, Ana!"`.
6. Crie `fibonacci(n)` com recursão (0, 1, 1, 2, 3, 5, 8...).
7. Crie `aplicarDuasVezes(fn, valor)` que aplique `fn` ao valor e depois ao resultado.

7. Arrays (listas)

Criando e acessando

Um array guarda uma lista ordenada de valores. As posições começam em **0**:

```
const frutas = ["maçã", "banana", "laranja"];
console.log(frutas[0]);        // "maçã"
console.log(frutas[2]);        // "laranja"
console.log(frutas.at(-1));    // "laranja" (-1 = último item)
console.log(frutas.length);    // 3
console.log(frutas[10]);       // undefined (posição inexistente)
```

Um array pode misturar tipos, mas em geral guardamos itens do mesmo tipo.

Adicionar, remover e alterar

```
const lista = ["a", "b", "c"];
lista.push("d");       // adiciona no fim     -> ["a","b","c","d"]
lista.unshift("z");    // adiciona no início  -> ["z","a","b","c","d"]
lista.pop();           // remove o último     -> ["z","a","b","c"]
lista.shift();         // remove o primeiro   -> ["a","b","c"]
lista[1] = "B";        // altera              -> ["a","B","c"]
// splice(posição, quantos remover, ...itens a inserir)
lista.splice(1, 1);            // remove 1 item na posição 1 -> ["a","c"]
lista.splice(1, 0, "x", "y");  // insere sem remover -> ["a","x","y","c"]
```

Atenção: `push`, `pop`, `shift`, `unshift`, `splice`, `sort` e `reverse` **modificam** o array original. `slice`, `map`, `filter`, `concat` e o operador spread criam um array **novo**.

Copiando e juntando

```
const original = [1, 2, 3];
const copia = [...original];             // spread: cópia rasa
const juntos = [...original, 4, 5];      // [1, 2, 3, 4, 5]
const parte = original.slice(0, 2);      // [1, 2] (fim não incluso)
const outro = original.concat([9, 10]);  // [1, 2, 3, 9, 10]
```

Procurando

```
const nums = [5, 12, 8, 130, 44];
nums.includes(8);                // true
nums.indexOf(130);               // 3 (ou -1 se não achar)
nums.find(n => n > 10);          // 12 (primeiro que atende)
nums.findIndex(n => n > 10);     // 1
nums.some(n => n > 100);         // true (algum atende?)
nums.every(n => n > 3);          // true (todos atendem?)
```

Percorrendo

```
const cores = ["azul", "verde", "rosa"];
for (const cor of cores) console.log(cor);
cores.forEach((cor, indice) => {
  console.log(indice, cor);
});
```

map, filter e reduce

Estes três métodos resolvem a maior parte dos problemas com listas, sem precisar de loops manuais.

```
const numeros = [1, 2, 3, 4, 5, 6];
// map: transforma cada item e devolve um array do mesmo tamanho
const dobros = numeros.map(n => n * 2);
console.log(dobros);   // [2, 4, 6, 8, 10, 12]
// filter: mantém apenas os itens que passam no teste
const pares = numeros.filter(n => n % 2 === 0);
console.log(pares);    // [2, 4, 6]
// reduce: combina todos os itens em um único valor
const soma = numeros.reduce((acumulado, n) => acumulado + n, 0);
console.log(soma);     // 21
```

No `reduce`, o segundo argumento (`0`) é o valor inicial do acumulador. Os métodos podem ser encadeados:

```
const resultado = numeros
  .filter(n => n % 2 === 0)   // [2, 4, 6]
  .map(n => n * 10)           // [20, 40, 60]
  .reduce((a, n) => a + n, 0);
console.log(resultado);       // 120
```

Ordenando

O `sort()` sem argumentos ordena como **texto**, o que dá resultados inesperados com números. Passe sempre uma função de comparação:

```
const n = [10, 9, 1, 100];
console.log([...n].sort());                // [1, 10, 100, 9]  (errado!)
console.log([...n].sort((a, b) => a - b)); // [1, 9, 10, 100]  crescente
console.log([...n].sort((a, b) => b - a)); // [100, 10, 9, 1]  decrescente
const nomes = ["Édson", "Ana", "Bruno"];
nomes.sort((a, b) => a.localeCompare(b, "pt-BR")); // respeita acentos
```

Como `sort` altera o original, o exemplo usa `[...n]` para ordenar uma cópia. Em ambientes recentes, `toSorted()` faz isso por você.

Outros métodos úteis

```
["a", "b", "c"].join("-");        // "a-b-c"
"a,b,c".split(",");               // ["a", "b", "c"]
[1, [2, [3]]].flat(Infinity);     // [1, 2, 3]
[1, 2, 3].reverse();              // [3, 2, 1]
Array.from({ length: 5 }, (_, i) => i + 1); // [1, 2, 3, 4, 5]
Array.isArray([]);                // true
```

Destructuring de arrays

```
const [primeiro, segundo, ...resto] = [10, 20, 30, 40];
console.log(primeiro, segundo, resto); // 10 20 [30, 40]
let a = 1, b = 2;
[a, b] = [b, a];                  // troca valores
console.log(a, b);                // 2 1
```

Resumo dos métodos

Método	O que faz	Altera o original?
push / pop	Adiciona ou remove no fim	Sim
unshift / shift	Adiciona ou remove no início	Sim
splice	Remove ou insere em qualquer posição	Sim
sort / reverse	Ordena ou inverte	Sim
slice	Copia um trecho	Não
map	Transforma cada item	Não
filter	Seleciona itens	Não
reduce	Reduz a um único valor	Não
find / includes	Busca itens	Não

Exercícios

1. Dado `[3, 8, 1, 9, 4]`, mostre o maior, o menor e a soma.
2. Crie `contarPares(lista)` usando `filter`.
3. Dado `["ana", "carlos", "bia"]`, gere um array com os nomes em maiúsculas usando `map`.
4. Remova os itens duplicados de uma lista (dica: `new Set`).
5. Crie `media(lista)` com `reduce`.
6. Ordene uma lista de palavras por tamanho, da menor para a maior.
7. Dado `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, calcule a soma dos quadrados dos números ímpares.

8. Objetos e dicionários

O que é um objeto

Um objeto guarda pares de **chave: valor**. É o "dicionário" do JavaScript e a estrutura mais usada da linguagem, porque representa coisas do mundo real (pessoa, produto, pedido).

```
const pessoa = {
  nome: "Ana",
  idade: 25,
  cidade: "Cabo Frio",
  hobbies: ["ler", "correr"],
  endereco: { rua: "Rua A", numero: 10 }
};
console.log(pessoa.nome);              // "Ana" (notação de ponto)
console.log(pessoa["idade"]);          // 25 (notação de colchetes)
console.log(pessoa.endereco.rua);      // "Rua A"
console.log(pessoa.hobbies[0]);        // "ler"
```

Use colchetes quando o nome da chave está em uma variável ou tem caracteres especiais:

```
const campo = "cidade";
console.log(pessoa[campo]);            // "Cabo Frio"
```

Alterar, adicionar e remover

```
pessoa.idade = 26;                 // altera
pessoa.profissao = "Dev";          // adiciona
delete pessoa.cidade;              // remove
console.log("nome" in pessoa);     // true
console.log(pessoa.cidade);        // undefined
```

Mesmo declarado com `const`, o conteúdo do objeto pode mudar. O que `const` impede é trocar a variável por outro objeto.

Métodos e this

Uma função guardada em um objeto é chamada de **método**. Dentro dele, `this` aponta para o próprio objeto:

```
const conta = {
  titular: "Ana",
  saldo: 100,
  depositar(valor) {
    this.saldo += valor;
    return this.saldo;
  }
};
console.log(conta.depositar(50)); // 150
```

Percorrendo objetos

```
const produto = { nome: "Mouse", preco: 59.9, estoque: 12 };
console.log(Object.keys(produto));    // ["nome", "preco", "estoque"]
console.log(Object.values(produto));  // ["Mouse", 59.9, 12]
console.log(Object.entries(produto)); // [["nome","Mouse"], ["preco",59.9], ...]
for (const [chave, valor] of Object.entries(produto)) {
  console.log(`${chave}: ${valor}`);
}
```

Destructuring, spread e atalhos

```
const { nome, preco } = produto;            // extrai propriedades
const { estoque: qtd } = produto;           // renomeia: qtd = 12
const atualizado = { ...produto, preco: 49.9 };  // cópia com alteração
const completo = { ...produto, categoria: "Periféricos" };
const x = 10, y = 20;
const ponto = { x, y };                     // atalho: { x: 10, y: 20 }
const chave = "cor";
const item = { [chave]: "azul" };           // chave calculada: { cor: "azul" }
```

O spread cria uma **cópia rasa**: objetos aninhados continuam compartilhados com o original. Para cópia completa use `structuredClone`:

```
const a = { dados: { n: 1 } };
const rasa = { ...a };
const profunda = structuredClone(a);
a.dados.n = 99;
console.log(rasa.dados.n);      // 99 (compartilha)
console.log(profunda.dados.n);  // 1  (independente)
```

Lista de objetos: a "tabela" do dia a dia

A estrutura mais comum em programas reais é um array de objetos. Combine com `filter`, `map` e `reduce`:

```
const alunos = [
  { nome: "Ana", nota: 8 },
  { nome: "Carlos", nota: 5 },
  { nome: "Bia", nota: 9 }
];
const aprovados = alunos.filter(a => a.nota >= 7);
const nomes = aprovados.map(a => a.nome);
const media = alunos.reduce((s, a) => s + a.nota, 0) / alunos.length;
console.log(nomes);             // ["Ana", "Bia"]
console.log(media.toFixed(1));  // "7.3"
```

Agrupar itens por uma propriedade é um padrão muito comum:

```
const compras = [
  { item: "Pão", tipo: "padaria" },
  { item: "Leite", tipo: "mercado" },
  { item: "Bolo", tipo: "padaria" }
];
const porTipo = {};
for (const c of compras) {
  if (!porTipo[c.tipo]) porTipo[c.tipo] = [];
  porTipo[c.tipo].push(c.item);
}
console.log(porTipo); // { padaria: ["Pão", "Bolo"], mercado: ["Leite"] }
```

JSON

JSON é um formato de texto para trocar dados, muito parecido com objetos JavaScript. Duas funções fazem a conversão:

```
const texto = JSON.stringify({ nome: "Ana", idade: 25 });
console.log(texto);             // '{"nome":"Ana","idade":25}'
const objeto = JSON.parse(texto);
console.log(objeto.nome);       // "Ana"
```

Em JSON, as chaves e os textos usam sempre aspas duplas, e não há funções nem `undefined`.

Map e Set

`Map` é um dicionário mais completo: aceita qualquer tipo de chave, mantém a ordem de inserção e conhece seu tamanho. `Set` guarda valores únicos.

```
const mapa = new Map();
mapa.set("a", 1);
mapa.set("b", 2);
console.log(mapa.get("a"));    // 1
console.log(mapa.has("c"));    // false
console.log(mapa.size);        // 2
for (const [k, v] of mapa) console.log(k, v);
const unicos = new Set([1, 2, 2, 3, 3, 3]);
console.log([...unicos]);      // [1, 2, 3]
```

> **Dica:** use objeto comum para dados estruturados com chaves fixas (pessoa, produto) e `Map` quando as chaves são dinâmicas, numerosas ou não são texto.

Exercícios

1. Crie um objeto `livro` com título, autor e ano e mostre cada propriedade.
2. Crie `apresentar(pessoa)` que receba um objeto e retorne `"Meu nome é X e tenho Y anos"`.
3. Crie `contarChaves(obj)` usando `Object.keys`.
4. Dado o array `alunos`, crie `melhorAluno(alunos)` que retorne o objeto com a maior nota.
5. Crie `contarLetras(texto)` que retorne um objeto como `{ o: 1, l: 1, a: 1 }` para `"ola"`.
6. Dado um array de produtos com `preco` e `quantidade`, calcule o valor total do estoque.
7. Converta um objeto em JSON e de volta, e confirme que os dados são iguais.
8. Use um `Map` para contar quantas vezes cada palavra aparece em uma frase.

9. Strings, números e datas

Métodos de string

Strings são imutáveis: os métodos devolvem uma nova string e não alteram a original.

```
const t = "  Aprendendo JavaScript  ";
t.length;                       // 25
t.trim();                       // "Aprendendo JavaScript"
t.toUpperCase();                // "  APRENDENDO JAVASCRIPT  "
t.toLowerCase();                // "  aprendendo javascript  "
t.includes("Java");             // true
t.startsWith("  Apr");          // true
t.indexOf("Script");            // 17
t.slice(2, 12);                 // "Aprendendo"
t.trim().split(" ");            // ["Aprendendo", "JavaScript"]
t.replace("Java", "Type");      // troca a primeira ocorrência
t.replaceAll("a", "4");         // troca todas
"5".padStart(3, "0");           // "005"
"ab".repeat(3);                 // "abababab"
"JS"[0];                        // "J" (acesso por posição)
```

Para inverter um texto, converta em array, inverta e junte de volta:

```
const inverter = s => s.split("").reverse().join("");
console.log(inverter("arara")); // "arara"
```

Expressões regulares (introdução)

Expressões regulares (regex) descrevem padrões de texto. São escritas entre barras:

```
const email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
console.log(email.test("ana@exemplo.com"));  // true
console.log(email.test("ana@"));             // false
"Tel: 21 99999-8888".match(/\d+/g);   // ["21", "99999", "8888"]
"a1b2c3".replace(/\d/g, "#");         // "a#b#c#"
```

O sufixo `g` (global) aplica a todas as ocorrências. Regex é um assunto grande; aprenda aos poucos e teste padrões em sites como regex101.com.

O objeto Math

```
Math.round(4.5);    // 5
Math.floor(4.9);    // 4 (arredonda para baixo)
Math.ceil(4.1);     // 5 (arredonda para cima)
Math.trunc(-4.9);   // -4 (remove a parte decimal)
Math.abs(-7);       // 7
Math.max(3, 9, 5);  // 9
Math.min(3, 9, 5);  // 3
Math.sqrt(81);      // 9
Math.PI;            // 3.141592653589793
// número aleatório inteiro entre min e max (inclusive)
const aleatorio = (min, max) =>
  Math.floor(Math.random() * (max - min + 1)) + min;
console.log(aleatorio(1, 6)); // simula um dado
```

Formatando números

```
(1234.5678).toFixed(2);   // "1234.57" (retorna texto)
const preco = 1234.5;
console.log(new Intl.NumberFormat("pt-BR", {
  style: "currency",
  currency: "BRL"
}).format(preco));         // "R$ 1.234,50"
console.log(preco.toLocaleString("pt-BR")); // "1.234,5"
```

Datas

```
const agora = new Date();
const natal = new Date(2026, 11, 25);     // ATENÇÃO: mês começa em 0 (11 = dezembro)
const iso = new Date("2026-12-25T10:30:00");
agora.getFullYear();    // ano
agora.getMonth();       // 0 a 11
agora.getDate();        // dia do mês
agora.getDay();         // 0 (domingo) a 6 (sábado)
agora.getHours();
console.log(agora.toLocaleDateString("pt-BR"));  // ex.: "01/11/2026"
console.log(agora.toLocaleTimeString("pt-BR"));  // ex.: "16:05:30"
// diferença em dias
const dias = Math.ceil((natal - agora) / (1000 * 60 * 60 * 24));
console.log(`Faltam ${dias} dias para o Natal`);
Date.now();             // milissegundos desde 1970 (timestamp)
```

> **Atenção:** o objeto `Date` tem várias armadilhas, como meses a partir de zero e fusos horários. Em projetos reais, bibliotecas como date-fns facilitam muito (capítulo 18).

Exercícios

1. Crie `ehPalindromo(texto)` ignorando maiúsculas e espaços.
2. Crie `capitalizar(frase)` que coloque a primeira letra de cada palavra em maiúscula.
3. Valide um CPF apenas pelo formato `000.000.000-00` usando regex.
4. Simule o lançamento de dois dados e mostre a soma.
5. Mostre o valor `4599.9` formatado em reais.
6. Calcule a sua idade em anos a partir da data de nascimento.
7. Mostre o dia da semana (por extenso, em português) de uma data qualquer.

PARTE II

Linguagem avançada

Tratamento de erros, módulos, classes, código assíncrono, manipulação de páginas web e as boas práticas dos profissionais.

10. Erros e depuração

try, catch e finally

Quando algo dá errado em tempo de execução, o JavaScript "lança" um erro e interrompe o programa. Com `try...catch` você captura o erro e decide o que fazer:

```
try {
  const dados = JSON.parse("{ isto não é JSON }");
  console.log(dados);
} catch (erro) {
  console.error("Falha ao ler os dados:", erro.message);
} finally {
  console.log("Executa sempre, com erro ou sem erro");
}
```

O bloco `finally` é opcional e serve para limpeza (fechar arquivos, esconder um "carregando").

Lançando seus próprios erros

Use `throw new Error(...)` quando uma função recebe dados que ela não consegue tratar:

```
function dividir(a, b) {
  if (typeof a !== "number" || typeof b !== "number") {
    throw new TypeError("Os dois argumentos devem ser números");
  }
  if (b === 0) {
    throw new RangeError("Divisão por zero");
  }
  return a / b;
}
try {
  dividir(10, 0);
} catch (e) {
  console.log(e.name, "-", e.message); // RangeError - Divisão por zero
}
```

Os tipos de erro mais comuns são `Error` (genérico), `TypeError` (tipo errado, ou chamar algo que não é função), `ReferenceError` (variável inexistente), `SyntaxError` (código ou JSON inválido) e `RangeError` (valor fora do intervalo).

Lendo mensagens de erro

Quando o erro não é capturado, o console mostra uma **stack trace**: a lista de chamadas que levou ao problema. Leia sempre a primeira linha (o tipo e a mensagem) e depois o primeiro arquivo seu que aparece na lista, com número de linha.

Mensagem	Causa provável
x is not defined	Variável inexistente ou com erro de digitação
Cannot read properties of undefined	Acessou uma propriedade de algo que é undefined
x is not a function	Chamou como função algo que não é
Unexpected token	Erro de sintaxe: falta fechar parêntese, chave ou aspas
Assignment to constant variable	Tentou reatribuir uma const

Técnicas de depuração

1. **console.log** estratégico: mostre valores nos pontos suspeitos. Use `console.log({ variavel })` para ver o nome junto com o valor.
2. **console.table** para arrays de objetos e **console.dir** para inspecionar objetos.
3. **debugger**: a palavra `debugger;` pausa a execução quando as ferramentas do navegador estão abertas.
4. **Breakpoints** nas DevTools (aba Sources): clique no número da linha, recarregue e avance passo a passo vendo o valor de cada variável.
5. **Isole o problema**: reduza o código até o menor exemplo que reproduz o erro.

> **Dica:** o método mais eficaz para achar bugs é explicar o código em voz alta, linha por linha, para si mesmo ou para um colega (o "patinho de borracha").

Exercícios

1. Escreva `lerJSON(texto)` que retorne o objeto ou `null` se o texto for inválido.
2. Crie `raizQuadrada(n)` que lance um erro para números negativos e teste com `try...catch`.
3. Provoque de propósito cada erro da tabela acima e leia a mensagem.
4. Use um breakpoint nas DevTools para acompanhar um loop que soma de 1 a 10.

11. Módulos

Por que dividir o código

Em projetos reais, o código fica separado em vários arquivos, cada um com uma responsabilidade. Os **módulos ES** (ES modules) são o sistema padrão do JavaScript para isso, com as palavras `export` e `import`.

Exportando e importando

Arquivo `matematica.js`:

```
export const PI = 3.14159;
export function somar(a, b) {
  return a + b;
}
export function area(raio) {
  return PI * raio ** 2;
}
// exportação padrão: apenas uma por arquivo
export default function saudar(nome) {
  return `Olá, ${nome}!`;
}
```

Arquivo `app.js`:

```
import saudar, { somar, area, PI } from "./matematica.js";
import { somar as soma } from "./matematica.js";   // renomeando
import * as mat from "./matematica.js";            // tudo em um objeto
console.log(saudar("Ana"));   // Olá, Ana!
console.log(somar(2, 3));     // 5
console.log(mat.area(2));     // 12.56636
```

A exportação **nomeada** exige as chaves e o mesmo nome no import. A exportação **padrão** (`default`) é importada sem chaves e com o nome que você quiser.

Usando módulos no navegador e no Node.js

No navegador, declare o script como módulo:

```
<script type="module" src="app.js"></script>
```

No Node.js, use a extensão `.mjs` ou adicione `"type": "module"` ao `package.json`:

```
{
  "name": "meu-projeto",
  "type": "module"
}
```

Você também encontrará o formato antigo do Node, o **CommonJS**, com `require` e `module.exports`. Ele ainda aparece em projetos e bibliotecas mais antigos, mas projetos novos usam ES modules.

```
// CommonJS (formato antigo)
const fs = require("fs");
module.exports = { somar };
```

Quando precisar carregar um módulo apenas sob demanda, use o **import dinâmico**, que retorna uma promessa:

```
const { somar } = await import("./matematica.js");
```

Exercícios

1. Crie `utils.js` com as funções `dobro`, `triplo` e `ehPar`, exportadas, e use-as em `app.js`.
2. Crie um módulo `formatar.js` com uma exportação padrão `formatarReais(valor)`.
3. Rode o projeto com Node.js usando `"type": "module"`.

12. Classes e programação orientada a objetos

Classes

Uma **classe** é um molde para criar objetos com a mesma estrutura e o mesmo comportamento:

```
class Pessoa {
  constructor(nome, idade) {
    this.nome = nome;
    this.idade = idade;
  }
  apresentar() {
    return `Oi, eu sou ${this.nome} e tenho ${this.idade} anos.`;
  }
  fazerAniversario() {
    this.idade++;
  }
}
const ana = new Pessoa("Ana", 25);
console.log(ana.apresentar());
ana.fazerAniversario();
console.log(ana.idade); // 26
```

O `constructor` roda uma vez, quando o objeto é criado com `new`. Os métodos ficam disponíveis em todas as instâncias.

Herança

Uma classe pode **estender** outra, reaproveitando e especializando seu comportamento:

```
class Estudante extends Pessoa {
  constructor(nome, idade, curso) {
    super(nome, idade);        // chama o constructor da classe pai
    this.curso = curso;
  }
  apresentar() {
    return `${super.apresentar()} Estudo ${this.curso}.`;
  }
}
const bia = new Estudante("Bia", 20, "Design");
console.log(bia.apresentar());
console.log(bia instanceof Pessoa);   // true
```

Campos privados, getters, setters e static

```
class ContaBancaria {
  #saldo = 0;                    // privado: só acessível dentro da classe
  static totalDeContas = 0;      // pertence à classe, não às instâncias
  constructor(titular) {
    this.titular = titular;
    ContaBancaria.totalDeContas++;
  }
  depositar(valor) {
    if (valor <= 0) throw new Error("Valor inválido");
    this.#saldo += valor;
  }
```

```
  get saldo() {                  // lido como propriedade: conta.saldo
    return this.#saldo;
  }
}
const c = new ContaBancaria("Ana");
c.depositar(100);
console.log(c.saldo);                    // 100
console.log(ContaBancaria.totalDeContas); // 1
// console.log(c.#saldo);                // ERRO de sintaxe: campo privado
```

O valor de this

O valor de `this` depende de **como** a função é chamada, não de onde foi escrita:

```
const pessoa = {
  nome: "Ana",
  falar() { return this.nome; }
};
console.log(pessoa.falar());         // "Ana" (this = pessoa)
const solta = pessoa.falar;
// console.log(solta());             // this perdido: erro ou undefined
const presa = pessoa.falar.bind(pessoa);
console.log(presa());                // "Ana" (bind fixa o this)
```

Funções de seta **não** têm `this` próprio: usam o `this` do lugar onde foram criadas. Por isso são ideais para callbacks dentro de métodos, e inadequadas como métodos de objeto.

Protótipos (visão geral)

Por baixo das classes, o JavaScript usa **protótipos**: cada objeto tem uma ligação com outro objeto de onde herda propriedades. `class` é uma sintaxe mais amigável sobre esse mecanismo. Quando você chama `ana.apresentar()`, o motor procura o método em `ana`, depois no protótipo de `Pessoa`, depois em `Object`, e assim por diante (a "cadeia de protótipos").

> **Dica:** prefira **composição** à herança profunda. Em vez de criar cadeias longas de classes, monte comportamentos juntando funções e objetos pequenos. Muitos projetos JavaScript usam poucas classes ou nenhuma.

Exercícios

1. Crie a classe `Retangulo` com `largura`, `altura` e métodos `area()` e `perimetro()`.
2. Crie `Quadrado` que estende `Retangulo` recebendo apenas o lado.
3. Crie `ContaBancaria` com `sacar`, `depositar` e saldo privado. Impeça saques maiores que o saldo.
4. Crie a classe `Carrinho` com uma lista de itens, métodos `adicionar`, `remover` e `total()`.

13. Código assíncrono

Por que existe

Algumas tarefas demoram: buscar dados na internet, ler arquivos, esperar o usuário. Se o JavaScript ficasse parado esperando, a página congelaria. Em vez disso, a linguagem inicia a tarefa, segue executando o resto e **volta depois** com o resultado. Isso é o código assíncrono.

```
console.log("1: começou");
setTimeout(() => {
  console.log("3: passou 1 segundo");
}, 1000);
console.log("2: continuou sem esperar");
```

A saída é 1, 2 e só depois 3.

Callbacks e o "inferno de callbacks"

Antigamente, o resultado chegava por funções de retorno. Várias tarefas em sequência levavam a código aninhado e difícil de ler:

```
buscarUsuario(1, usuario => {
  buscarPedidos(usuario.id, pedidos => {
    buscarItens(pedidos[0].id, itens => {
      console.log(itens);   // pirâmide de aninhamento
    });
  });
});
```

Promises

Uma **Promise** (promessa) representa um resultado futuro. Ela está em um de três estados: pendente, cumprida (`resolve`) ou rejeitada (`reject`).

```
const esperar = ms =>
  new Promise(resolve => setTimeout(resolve, ms));
function dividirAsync(a, b) {
  return new Promise((resolve, reject) => {
    if (b === 0) reject(new Error("Divisão por zero"));
    else resolve(a / b);
  });
}
dividirAsync(10, 2)
  .then(resultado => console.log("Resultado:", resultado))
  .catch(erro => console.error("Erro:", erro.message))
  .finally(() => console.log("Fim"));
```

async e await

`async/await` permite escrever código assíncrono com cara de código normal, de cima para baixo. Uma função `async` sempre retorna uma Promise, e `await` espera uma Promise terminar:

```
async function principal() {
  try {
    console.log("Esperando...");
    await esperar(1000);
    const resultado = await dividirAsync(10, 2);
    console.log("Resultado:", resultado);
  } catch (erro) {
    console.error("Deu erro:", erro.message);
  }
}
principal();
```

Use `try...catch` para tratar erros de funções `await`. Em módulos ES, o `await` também funciona no topo do arquivo, fora de funções.

fetch: buscando dados na internet

A função `fetch` faz requisições HTTP e funciona no navegador e no Node.js:

```
async function buscarUsuarios() {
  try {
    const resposta = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!resposta.ok) {
      throw new Error(`Erro HTTP ${resposta.status}`);
    }
    const usuarios = await resposta.json();
    for (const u of usuarios) {
      console.log(u.name, "-", u.email);
    }
  } catch (erro) {
    console.error("Não foi possível buscar:", erro.message);
  }
}
buscarUsuarios();
```

Atenção: o `fetch` só rejeita a Promise em falhas de rede. Respostas como 404 ou 500 **não** lançam erro: por isso verificamos `resposta.ok`.

Para enviar dados (POST):

```
const resposta = await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Olá", body: "Texto", userId: 1 })
});
const criado = await resposta.json();
console.log(criado);
```

Várias tarefas ao mesmo tempo

Quando as tarefas não dependem entre si, execute-as em paralelo:

```
const [users, posts] = await Promise.all([
  fetch("https://jsonplaceholder.typicode.com/users").then(r => r.json()),
  fetch("https://jsonplaceholder.typicode.com/posts").then(r => r.json())
]);
console.log(users.length, posts.length);
```

Método	Comportamento
Promise.all	Espera todas; rejeita se qualquer uma falhar
Promise.allSettled	Espera todas; informa sucesso ou falha de cada uma
Promise.race	Termina quando a primeira terminar
Promise.any	Termina quando a primeira for cumprida com sucesso

> **Atenção:** um erro comum é usar `await` dentro de um `for` quando as tarefas poderiam rodar juntas. Isso as executa uma por vez e deixa o programa mais lento.

O event loop (visão geral)

O JavaScript executa uma coisa por vez (uma única linha de execução), mas parece fazer várias porque delega tarefas demoradas ao navegador ou ao Node.js. Quando elas terminam, os resultados entram em filas e o **event loop** os executa assim que a pilha de chamadas fica livre. As Promises (microtarefas) têm prioridade sobre `setTimeout` (macrotarefas):

```
console.log("A");
setTimeout(() => console.log("B (timeout)"), 0);
Promise.resolve().then(() => console.log("C (promise)"));
console.log("D");
// Ordem: A, D, C, B
```

Exercícios

1. Crie `esperar(ms)` e use-a para mostrar uma contagem regressiva de 3 a 1.
2. Busque `https://jsonplaceholder.typicode.com/todos` e mostre quantas tarefas estão concluídas.
3. Busque os dados de um usuário e depois os posts desse usuário (duas chamadas em sequência).
4. Use `Promise.all` para buscar três recursos ao mesmo tempo e compare o tempo com a versão em sequência (`console.time`).
5. Preveja a ordem de saída do exemplo do event loop antes de rodar.

14. DOM e eventos (JavaScript no navegador)

O que é o DOM

O **DOM** (Document Object Model) é a representação da página como uma árvore de objetos. Com JavaScript você lê e modifica essa árvore, e o navegador atualiza a tela. O objeto `document` é a porta de entrada.

Selecionando elementos

```
const titulo = document.querySelector("h1");              // primeiro que combina
const botao = document.querySelector("#enviar");          // por id
const itens = document.querySelectorAll(".item");         // todos com a classe
const caixa = document.getElementById("caixa");           // alternativa por id
itens.forEach(el => console.log(el.textContent));
```

Os seletores seguem as mesmas regras do CSS: `.classe`, `#id`, `tag`, `div > p`, `[data-tipo="a"]`.

Lendo e alterando

```
titulo.textContent = "Novo título";            // texto puro (seguro)
caixa.innerHTML = "<strong>Olá</strong>";      // HTML (cuidado, veja abaixo)
caixa.classList.add("ativo");
caixa.classList.remove("ativo");
caixa.classList.toggle("escuro");              // liga e desliga
caixa.style.backgroundColor = "lightyellow";
botao.setAttribute("disabled", "");
console.log(botao.getAttribute("id"));
console.log(caixa.dataset.usuario);            // lê data-usuario="..."
```

> **Atenção:** nunca coloque texto digitado por usuários em `innerHTML`. Isso permite ataques de XSS (injeção de scripts). Use `textContent` para texto ou crie elementos com `createElement`.

Criando e removendo elementos

```
const lista = document.querySelector("#lista");
const li = document.createElement("li");
li.textContent = "Novo item";
li.classList.add("item");
lista.append(li);                 // adiciona no fim
lista.prepend(li.cloneNode(true)); // adiciona no início (cópia)
li.remove();                      // remove do documento
```

Eventos

Eventos são acontecimentos na página: clique, tecla pressionada, envio de formulário, carregamento. Você registra uma função para reagir a eles:

```
botao.addEventListener("click", evento => {
  console.log("Clicou!", evento.target);
});
document.addEventListener("keydown", e => {
  if (e.key === "Escape") console.log("ESC pressionado");
});
const form = document.querySelector("form");
form.addEventListener("submit", e => {
  e.preventDefault();             // impede o recarregamento da página
  const dados = new FormData(form);
  console.log(Object.fromEntries(dados));
});
```

Eventos comuns: `click`, `dblclick`, `input` (a cada tecla digitada), `change`, `submit`, `keydown`, `mouseover`, `scroll`, `DOMContentLoaded`.

Delegação de eventos

Em vez de registrar um evento em cada item de uma lista (inclusive nos que serão criados depois), registre um só no elemento pai e descubra qual filho foi clicado:

```
lista.addEventListener("click", e => {
  const item = e.target.closest("li");
  if (!item) return;
  item.classList.toggle("feito");
});
```

Guardando dados no navegador

O `localStorage` guarda pares chave-valor de texto que sobrevivem ao fechamento da página:

```
localStorage.setItem("tema", "escuro");
console.log(localStorage.getItem("tema"));   // "escuro"
localStorage.removeItem("tema");
// objetos e arrays precisam virar texto
const tarefas = [{ texto: "Estudar JS", feita: false }];
localStorage.setItem("tarefas", JSON.stringify(tarefas));
const salvas = JSON.parse(localStorage.getItem("tarefas")) ?? [];
```

Projeto: lista de tarefas completa

Este exemplo junta DOM, eventos, arrays e `localStorage`. Primeiro o HTML (`index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Tarefas</title>
  <style>
    .feita { text-decoration: line-through; color: gray; }
    li { cursor: pointer; margin: 4px 0; }
  </style>
</head>
<body>
  <h1>Minhas tarefas</h1>
  <form id="form">
    <input id="texto" placeholder="Nova tarefa" required>
    <button>Adicionar</button>
  </form>
  <ul id="lista"></ul>
```

```
  <script type="module" src="tarefas.js"></script>
</body>
</html>
```

Agora o JavaScript (`tarefas.js`):

```
const form = document.querySelector("#form");
const campo = document.querySelector("#texto");
const lista = document.querySelector("#lista");
let tarefas = JSON.parse(localStorage.getItem("tarefas")) ?? [];
function salvar() {
  localStorage.setItem("tarefas", JSON.stringify(tarefas));
}
function desenhar() {
  lista.innerHTML = "";
  tarefas.forEach((t, indice) => {
    const li = document.createElement("li");
    li.textContent = t.texto;
    li.classList.toggle("feita", t.feita);
    li.dataset.indice = indice;
    lista.append(li);
  });
}
form.addEventListener("submit", e => {
  e.preventDefault();
  tarefas.push({ texto: campo.value.trim(), feita: false });
  campo.value = "";
  salvar();
  desenhar();
});
lista.addEventListener("click", e => {
  const li = e.target.closest("li");
  if (!li) return;
  const t = tarefas[Number(li.dataset.indice)];
  t.feita = !t.feita;
  salvar();
  desenhar();
});
desenhar();
```

Dica: como módulos exigem um servidor, abra o projeto com a extensão Live Server do VS Code ou com `npx serve` no terminal, em vez de abrir o arquivo direto.

Repare no padrão: os **dados** (array `tarefas`) são a fonte da verdade, e a função `desenhar()` apenas reflete esses dados na tela. É a mesma ideia por trás de frameworks como React.

Exercícios

1. Crie um botão que aumenta um contador mostrado na página.
2. Crie um botão que alterna entre tema claro e escuro e lembre a escolha com `localStorage`.
3. Faça um formulário de cadastro com validação (nome obrigatório, e-mail válido) que mostre mensagens de erro na página.
4. Adicione ao projeto de tarefas um botão para excluir cada item e um contador de tarefas pendentes.
5. Crie uma página que busque dados de uma API e mostre os resultados em cards.

15. Boas práticas e ferramentas

Escrevendo código legível

- **Nomes claros**: `precoComDesconto` é melhor que `p2`. Funções começam com verbo (`calcularTotal`), booleanos com `is`, `tem` ou `pode` (`estaLogado`).
- **Funções pequenas**: cada função faz uma coisa e cabe na tela.
- **const por padrão**, `let` só quando precisar reatribuir, `var` nunca.
- **`===`** em vez de `==`.
- **Evite números e textos mágicos**: troque `if (status === 3)` por uma constante com nome.
- **Retorne cedo** para evitar muitos `if` aninhados.
- **Não repita código** (princípio DRY): se copiou e colou três vezes, crie uma função.
- **Comente o porquê**, não o óbvio. O código mostra o "como"; o comentário explica a razão.

Prettier e ESLint

Duas ferramentas melhoram a qualidade sem esforço:

- **Prettier** formata o código automaticamente (espaços, aspas, quebras de linha), acabando com discussões de estilo.
- **ESLint** analisa o código e aponta erros prováveis, como variáveis não usadas ou comparações perigosas.

```
npm install --save-dev prettier eslint
npx prettier --write .
```

No VS Code, ative "Format on Save" nas configurações para formatar toda vez que salvar.

Controle de versão com Git

Git registra o histórico do seu código e permite voltar atrás. O GitHub (e serviços parecidos) guarda o projeto na nuvem. Comandos essenciais:

```
git init                      # começa a versionar a pasta
git add .                     # prepara as mudanças
git commit -m "Adiciona lista de tarefas"
git status                    # vê o que mudou
git log --oneline             # histórico resumido
git branch nova-funcao        # cria uma ramificação
git switch nova-funcao        # troca de ramificação
git push                      # envia ao repositório remoto
```

Crie um arquivo `.gitignore` com `node_modules` e arquivos de senha, para que eles nunca sejam enviados.

Segurança básica

- Nunca confie em dados vindos do usuário: valide e trate sempre.
- Evite `innerHTML` com texto de usuário (XSS) e nunca use `eval`.

- **Nunca** coloque senhas, chaves de API ou tokens no código do navegador: qualquer pessoa pode ler. Segredos ficam no servidor, em variáveis de ambiente.
- Mantenha as dependências atualizadas e rode `npm audit` de vez em quando.

Desempenho básico

- Evite repetir trabalho pesado dentro de loops.
- Use **debounce** em eventos que disparam muitas vezes (digitar, rolar a tela): espere o usuário parar antes de agir.
- Manipule o DOM em lote em vez de elemento por elemento.
- Meça antes de otimizar: use a aba Performance das DevTools.

```
function debounce(fn, espera = 300) {
  let temporizador;
  return (...args) => {
    clearTimeout(temporizador);
    temporizador = setTimeout(() => fn(...args), espera);
  };
}
const buscar = debounce(termo => console.log("Buscando", termo), 400);
campoBusca.addEventListener("input", e => buscar(e.target.value));
```

Como aprender mais rápido

- Leia a documentação do **MDN** (developer.mozilla.org): é a referência mais confiável de JavaScript.
- Aprenda a pesquisar a mensagem de erro exata.
- Construa projetos que você realmente queira usar.
- Leia código de outras pessoas e revise o seu depois de alguns dias.
- Ensine o que aprendeu: escrever um texto ou explicar a um colega consolida o conhecimento.

Exercícios

1. Pegue um código seu de capítulos anteriores e melhore os nomes das variáveis e funções.
2. Instale e configure o Prettier no seu projeto.
3. Crie um repositório Git para seus exercícios e faça ao menos três commits.
4. Implemente uma busca com debounce que filtre uma lista enquanto o usuário digita.

PARTE III

Ecossistema e bibliotecas

Node.js, npm e as bibliotecas que profissionais usam todos os dias, do front-end ao back-end, incluindo testes e TypeScript.

16. Node.js e npm

O que é o Node.js

O Node.js é um ambiente que executa JavaScript fora do navegador. Ele usa o mesmo motor de JavaScript do Chrome (V8) e adiciona recursos que o navegador não tem: ler e gravar arquivos, criar servidores, acessar o sistema operacional. Com ele você escreve APIs, scripts de automação, ferramentas de linha de comando e sites completos.

Diferenças em relação ao navegador:

Recurso	Navegador	Node.js
document, window (DOM)	Sim	Não
Arquivos do computador (fs)	Não	Sim
Criar servidor HTTP	Não	Sim
fetch, setTimeout, Promise	Sim	Sim
Módulos ES (import)	Sim	Sim

Arquivos, caminhos e variáveis de ambiente

Os módulos nativos do Node (importados com o prefixo `node:`) cobrem tarefas comuns:

```
import { readFile, writeFile } from "node:fs/promises";
import path from "node:path";
const arquivo = path.join("dados", "nota.txt");   // monta o caminho certo no SO
await writeFile("nota.txt", "Olá, arquivo!", "utf8");
const texto = await readFile("nota.txt", "utf8");
console.log(texto);
console.log(process.argv.slice(2));        // argumentos: node app.js a b -> ["a", "b"]
console.log(process.env.PORT ?? 3000);     // variável de ambiente com padrão
```

Um servidor HTTP mínimo

```
import { createServer } from "node:http";
const servidor = createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });
  res.end("Olá do servidor!");
});
servidor.listen(3000, () => {
  console.log("Rodando em http://localhost:3000");
});
```

Rode com `node servidor.js` e abra o endereço no navegador. Na prática, você usará uma biblioteca como o Express (capítulo 20) para facilitar rotas e respostas.

npm: o gerenciador de pacotes

O **npm** vem com o Node.js e é o maior repositório de código aberto do mundo. Ele instala bibliotecas (chamadas de **pacotes**) e gerencia suas versões. Comandos essenciais:

```
npm init -y                 # cria o package.json
npm install lodash          # instala e registra como dependência
npm install -D prettier     # dependência só de desenvolvimento
npm uninstall lodash        # remove
npm install                 # instala tudo que está no package.json
npm run dev                 # executa um script definido no package.json
npx prettier --write .      # executa uma ferramenta sem instalar globalmente
npm outdated                # lista pacotes desatualizados
npm audit                   # verifica vulnerabilidades conhecidas
```

O package.json

É o arquivo de identidade do projeto: nome, scripts e dependências.

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node src/index.js",
    "dev": "node --watch src/index.js",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^5.0.0"
  },
  "devDependencies": {
    "prettier": "^3.0.0"
  }
}
```

- `scripts` define atalhos: `npm run dev`, `npm start`, `npm test`.
- `dependencies` são necessárias para o programa rodar; `devDependencies` só no desenvolvimento (testes, formatação).
- O prefixo `^` aceita atualizações compatíveis (`^1.2.3` permite 1.x.x mais novas); `~` aceita só correções (`~1.2.3` permite 1.2.x). Isso se chama versionamento semântico (**semver**): MAJOR.MINOR.PATCH.
- A pasta `node_modules` guarda os pacotes instalados e **nunca** vai para o Git.
- O `package-lock.json` trava as versões exatas: envie-o ao Git.

> **Atenção:** os números de versão do exemplo são ilustrativos. Ao instalar com `npm install`, o npm grava as versões atuais para você.

Alternativas ao npm

Outros gerenciadores fazem o mesmo trabalho com foco em velocidade ou economia de espaço, e usam o mesmo ecossistema de pacotes: **pnpm**, **Yarn** e o **Bun** (que também é um ambiente de execução). Você vai encontrar todos em projetos reais; os conceitos são iguais.

Exercícios

1. Crie uma pasta, rode `npm init -y` e adicione um script `"start"` que execute seu programa.
2. Escreva um script que leia um arquivo de texto e mostre quantas palavras ele tem.
3. Crie um programa que receba um nome por argumento (`node app.js Ana`) e imprima uma saudação.

4. Crie um servidor HTTP que responda com a hora atual.
5. Instale um pacote, use-o no código e depois localize-o em `node_modules` e no `package.json`.

17. Entendendo bibliotecas

Biblioteca, framework e runtime

Termo	Ideia	Exemplos
Biblioteca	Conjunto de funções que você chama quando quer	Lodash, date-fns, Axios
Framework	Estrutura que define como organizar o projeto e chama o seu código	Angular, Next.js, NestJS
Runtime	Ambiente que executa o JavaScript	Navegador, Node.js, Deno, Bun

Uma frase ajuda a lembrar: com uma **biblioteca**, você chama o código dela; com um **framework**, ele chama o seu código. React fica no meio do caminho: define a forma de construir interfaces, mas deixa muitas escolhas ao seu cargo.

Três formas de usar uma biblioteca

1. **CDN**: uma tag `<script>` com o endereço da biblioteca. Rápido para testar e aprender.
2. **npm + import**: instala com `npm install` e importa no código. É o jeito padrão em projetos.
3. **Bundler/ferramenta de build** (Vite e outros): reúne todos os arquivos e pacotes em arquivos otimizados para o navegador.

```html
<!-- 1) CDN (para testes rápidos) -->
<script src="https://cdn.jsdelivr.net/npm/lodash@4/lodash.min.js"></script>
<script>
  console.log(_.chunk([1, 2, 3, 4, 5], 2));
</script>
```

```bash
# 2) npm
npm install lodash
```

```js
// no código
import _ from "lodash";
console.log(_.chunk([1, 2, 3, 4, 5], 2)); // [[1, 2], [3, 4], [5]]
```

Vite: o ponto de partida dos projetos front-end

O **Vite** cria um projeto pronto com servidor de desenvolvimento rápido e build para produção. Funciona com JavaScript puro, React, Vue, Svelte e outros:

```bash
npm create vite@latest meu-app
cd meu-app
npm install
npm run dev            # abre o servidor de desenvolvimento
npm run build          # gera a versão final na pasta dist
```

Como escolher uma biblioteca

Antes de instalar, faça estas perguntas:

- **Preciso mesmo dela?** Muitas tarefas já têm solução nativa na linguagem.

- **Está mantida?** Veja a data do último lançamento e se há atividade no repositório.
- **É popular?** Muitos downloads semanais e estrelas no GitHub indicam uma comunidade ativa e mais respostas na internet.
- **A documentação é boa?** Exemplos claros economizam horas.
- **Qual o tamanho?** No navegador, cada kilobyte pesa no tempo de carregamento.
- **Qual a licença?** MIT e Apache 2.0 são permissivas; confira para uso comercial.
- **É segura?** Pacotes são código de terceiros que rodam no seu projeto. Desconfie de nomes parecidos com pacotes famosos e de pacotes sem histórico.

Uma boa fonte para pesquisar é o próprio site npmjs.com, que mostra downloads, versões, dependências e o link do repositório.

> **Dica:** pense em cada dependência como um compromisso de longo prazo. Poucas dependências bem escolhidas dão menos trabalho do que dezenas de pacotes pequenos.

Exercícios

1. Pesquise `date-fns` no npmjs.com e anote: último lançamento, downloads semanais e licença.
2. Use o Lodash via CDN em uma página HTML e chame `_.shuffle` em uma lista.
3. Crie um projeto com `npm create vite@latest` usando o modelo de JavaScript puro e rode-o.
4. Escolha uma tarefa (formatar datas, gerar IDs, validar e-mails) e compare duas bibliotecas que a resolvem.

18. Bibliotecas utilitárias

Estas bibliotecas funcionam em qualquer projeto, no navegador ou no Node.js, e resolvem problemas que aparecem o tempo todo.

Lodash: funções para dados

O **Lodash** reúne centenas de funções para arrays, objetos e textos. Tornou-se menos necessário porque a linguagem ganhou muitos recursos, mas segue muito usado em projetos existentes e ainda ajuda em casos como cópia profunda e agrupamentos.

```
import _ from "lodash";
const pessoas = [
  { nome: "Ana", cidade: "Cabo Frio", idade: 25 },
  { nome: "Bia", cidade: "Niterói", idade: 31 },
  { nome: "Caio", cidade: "Cabo Frio", idade: 19 }
];
_.groupBy(pessoas, "cidade");     // { "Cabo Frio": [Ana, Caio], "Niterói": [Bia] }
_.sortBy(pessoas, "idade");       // da mais nova para a mais velha
_.uniq([1, 2, 2, 3, 3]);          // [1, 2, 3]
_.chunk([1, 2, 3, 4, 5], 2);      // [[1, 2], [3, 4], [5]]
_.get(pessoas, "[0].endereco.rua", "sem rua");  // acesso seguro, com padrão
_.cloneDeep(pessoas);             // cópia profunda
const buscar = _.debounce(termo => console.log(termo), 300);
```

Para reduzir o tamanho no navegador, prefira importar só o que usa, a partir do pacote `lodash-es`:

```
import { groupBy, debounce } from "lodash-es";
```

date-fns: datas sem dor de cabeça

O **date-fns** oferece funções simples para formatar, comparar e calcular datas, sem alterar o objeto original. Alternativas parecidas: **Day.js** (leve, API semelhante à do antigo Moment.js) e **Luxon** (muito bom com fusos horários).

```
import { format, addDays, differenceInDays, isBefore } from "date-fns";
import { ptBR } from "date-fns/locale";
const hoje = new Date();
format(hoje, "dd/MM/yyyy");                          // "01/11/2026"
format(hoje, "EEEE, d 'de' MMMM", { locale: ptBR }); // "domingo, 1 de novembro"
const daqui7 = addDays(hoje, 7);
differenceInDays(daqui7, hoje);                      // 7
isBefore(hoje, daqui7);                              // true
```

> **Atenção:** no `format`, as letras têm significado próprio e diferenciam maiúsculas de minúsculas: `MM` é mês e `mm` é minuto; `yyyy` é o ano e `dd` é o dia.

Axios: requisições HTTP

O **Axios** é uma alternativa ao `fetch` com recursos prontos: converte JSON automaticamente, lança erro para respostas 4xx e 5xx, permite configurar uma instância com endereço base e tempo limite, e tem interceptadores.

```
import axios from "axios";
// requisição simples
const { data } = await axios.get("https://jsonplaceholder.typicode.com/posts", {
  params: { _limit: 5 }
});
console.log(data.length); // 5
// instância configurada, reaproveitada no projeto inteiro
const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
  timeout: 5000
});
try {
  const resp = await api.post("/posts", { title: "Olá", userId: 1 });
  console.log(resp.status, resp.data);
} catch (erro) {
  console.error(erro.response?.status ?? "sem resposta", erro.message);
}
```

Hoje o `fetch` nativo atende a maioria dos casos; escolha o Axios quando valer a pena pelos recursos extras.

Zod: validação de dados

Dados que chegam de formulários, APIs e arquivos nunca devem ser tratados como confiáveis. O **Zod** descreve o formato esperado e valida:

```
import { z } from "zod";
const Usuario = z.object({
  nome: z.string().min(2, "Nome muito curto"),
  email: z.string().email("E-mail inválido"),
  idade: z.number().int().min(18, "Apenas maiores de idade")
});
const resultado = Usuario.safeParse({
  nome: "Ana",
  email: "ana@exemplo.com",
  idade: 25
});
if (resultado.success) {
  console.log("Dados válidos:", resultado.data);
} else {
  console.log("Problemas:", resultado.error.issues);
}
```

Mais bibliotecas úteis, por tarefa

Tarefa	Opções populares
Gerar IDs únicos	crypto.randomUUID() (nativo), nanoid
Datas	date-fns, Day.js, Luxon
Dinheiro e decimais precisos	decimal.js, big.js
Ler e gerar CSV	PapaParse
Processar imagens no Node	sharp
Extrair dados de HTML	cheerio
Converter Markdown em HTML	marked, markdown-it
Cores no terminal	chalk, picocolors
Ler argumentos de linha de comando	commander, yargs
Variáveis de ambiente	dotenv, ou node --env-file
Programar tarefas agendadas	node-cron
Gráficos	Chart.js, D3, ECharts

Quando o nativo basta

Antes de instalar, verifique se o JavaScript já resolve:

Em vez de	Considere usar
jQuery	document.querySelector, classList, addEventListener, fetch
Moment.js (em manutenção apenas)	Intl.DateTimeFormat ou date-fns
Lodash (muitas funções)	map, filter, reduce, flat, structuredClone, Object.entries
uuid	crypto.randomUUID()
Axios (casos simples)	fetch
left-pad e afins	padStart, padEnd

Exercícios

1. Use `groupBy` do Lodash para agrupar uma lista de produtos por categoria e depois refaça com `reduce`.
2. Formate a data de hoje no padrão brasileiro e por extenso usando date-fns.
3. Consuma uma API pública com Axios, tratando erros de rede e de resposta.
4. Crie um esquema Zod para um cadastro de produto (nome, preço positivo, estoque inteiro) e teste dados válidos e inválidos.
5. Escolha uma biblioteca da tabela, leia o início da documentação e escreva um exemplo de 10 linhas.

19. Bibliotecas e frameworks de front-end

Por que usar um framework de interface

Em projetos pequenos, o DOM puro dá conta. Quando a tela tem muitos elementos que mudam conforme os dados, atualizar o DOM manualmente vira uma fonte de bugs. Bibliotecas de interface resolvem isso com uma ideia central: **você descreve como a tela deve ficar para um certo estado, e a biblioteca atualiza o DOM quando o estado muda**.

React

O **React** é a biblioteca de interface mais usada. Ele organiza a tela em **componentes**: funções que recebem dados (**props**) e devolvem a interface descrita em **JSX**, uma mistura de JavaScript com marcação parecida com HTML. Os dados que mudam ao longo do tempo ficam em **estado**, criado com o hook `useState`.

Para começar um projeto:

```
npm create vite@latest meu-app -- --template react
cd meu-app
npm install
npm run dev
```

Um componente com estado:

```
import { useState } from "react";
function Contador() {
  const [total, setTotal] = useState(0);
  return (
    <button onClick={() => setTotal(total + 1)}>
      Cliques: {total}
    </button>
  );
}
export default Contador;
```

Componentes recebem props e podem renderizar listas com `map`. Cada item precisa de uma `key` única e estável:

```
function ListaDeTarefas({ tarefas }) {
  return (
    <ul>
      {tarefas.map(t => (
        <li key={t.id} className={t.feita ? "feita" : ""}>
          {t.texto}
        </li>
      ))}
    </ul>
  );
}
```

Para buscar dados quando o componente aparece, use o hook `useEffect`:

```
import { useEffect, useState } from "react";
function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(r => r.json())
      .then(setUsuarios);
  }, []);   // lista vazia: executa só uma vez, ao montar
  return (
    <ul>
      {usuarios.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Atenção: no React, nunca altere o estado diretamente (`tarefas.push(...)`). Crie um novo valor e passe para a função de atualização: `setTarefas([...tarefas, nova])`. O React só redesenha a tela quando vê um valor novo.

Para aprender React com solidez, você já precisa dominar funções, arrays (`map`, `filter`), objetos, destructuring e assincronismo, exatamente o que este guia cobre.

Vue

O **Vue** é conhecido pela curva de aprendizado suave e por separar bem o HTML da lógica. Pode ser usado direto em uma página, sem ferramentas de build:

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<div id="app">
  <button @click="total++">Cliques: {{ total }}</button>
</div>
<script>
  const { createApp, ref } = Vue;
  createApp({
    setup() {
      const total = ref(0);
      return { total };
    }
  }).mount("#app");
</script>
```

Outras opções de interface

- **Svelte**: compila os componentes em JavaScript enxuto, com sintaxe muito curta.
- **Angular**: framework completo mantido pelo Google, com TypeScript, injeção de dependência e tudo integrado. Comum em empresas grandes.
- **Solid**: parecido com o React na escrita, com reatividade mais fina.
- **jQuery**: biblioteca antiga que simplificava o DOM. Está em milhões de sites antigos e você pode encontrá-la em manutenção, mas hoje o JavaScript nativo faz o mesmo.

Frameworks de aplicação (meta-frameworks)

Eles envolvem uma biblioteca de interface e adicionam rotas por pastas, renderização no servidor, otimizações e deploy simplificado:

Meta-framework	Base	Para que serve
Next.js	React	Sites e apps full-stack, SEO, renderização no servidor
Nuxt	Vue	O equivalente do Next.js para Vue
SvelteKit	Svelte	Apps completos com Svelte
Astro	Vários	Sites de conteúdo muito rápidos (blogs, documentação)
React Router / Remix	React	Rotas e carregamento de dados

O ecossistema ao redor

Uma aplicação real costuma combinar a biblioteca de interface com pacotes especializados:

Necessidade	Opções comuns
Rotas	React Router, Vue Router, rotas do Next.js
Estado global	Zustand, Redux Toolkit, Pinia (Vue)
Dados do servidor (cache, recarga)	TanStack Query
Formulários	React Hook Form, VeeValidate
Componentes prontos	shadcn/ui, Material UI, Chakra UI, Vuetify
Estilos	Tailwind CSS, Bootstrap, CSS Modules
Animações	Framer Motion, GSAP

Gráficos e visualização de dados

O **Chart.js** cria gráficos comuns (barras, linhas, pizza) com pouca configuração:

```
<canvas id="grafico" width="400" height="200"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script>
  new Chart(document.getElementById("grafico"), {
    type: "bar",
    data: {
      labels: ["Jan", "Fev", "Mar"],
      datasets: [{ label: "Vendas", data: [12, 19, 7] }]
    }
  });
</script>
```

Para visualizações sob medida, o **D3** liga dados a elementos SVG com total controle. Outras opções: **ECharts** (muito completo), **Leaflet** (mapas) e **Three.js** (gráficos 3D e jogos no navegador).

Fora do navegador

O mesmo conhecimento de JavaScript leva você a aplicativos de celular com **React Native** e **Expo**, e a programas de desktop com **Electron** (usado em editores e aplicativos de mensagens) ou **Tauri**.

Como escolher por onde começar

Se o objetivo é conseguir emprego, o React tem o maior número de vagas. Se você prefere uma entrada mais amigável, experimente o Vue ou o Svelte. O mais importante é dominar o JavaScript antes: quem entende a linguagem aprende qualquer framework mais rápido, e frameworks vêm e vão.

Exercícios

1. Recrie o projeto de tarefas do capítulo 14 usando React: um componente `App`, um `Lista` e um `Formulario`.
2. Crie um componente que busque usuários de uma API e permita filtrar por nome.
3. Faça a versão do contador com Vue via CDN.
4. Crie um gráfico de barras com Chart.js a partir de um array de objetos (use `map` para extrair rótulos e valores).
5. Monte um projeto com Vite, instale o React Router e crie duas páginas.

20. Back-end com Node.js

O que o back-end faz

O back-end é a parte do sistema que roda em um servidor: guarda dados, aplica regras de negócio, controla quem pode fazer o quê e responde às requisições do front-end. A comunicação costuma usar uma **API REST**, em que cada recurso (tarefas, usuários, produtos) tem um endereço e as ações são os métodos HTTP:

Método	Ação	Exemplo
GET	Ler	GET /tarefas lista as tarefas
POST	Criar	POST /tarefas cria uma tarefa
PUT / PATCH	Atualizar	PATCH /tarefas/3 altera a tarefa 3
DELETE	Remover	DELETE /tarefas/3 apaga a tarefa 3

Os códigos de status indicam o resultado: 200 (ok), 201 (criado), 400 (requisição inválida), 401 (não autenticado), 403 (sem permissão), 404 (não encontrado) e 500 (erro no servidor).

Express

O **Express** é o framework web mais conhecido do Node.js. Ele simplifica rotas, leitura do corpo das requisições e respostas. Instale com `npm install express` e use `"type": "module"` no `package.json`:

```
import express from "express";
const app = express();
app.use(express.json());          // lê corpos JSON das requisições
// middleware: roda em toda requisição
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();                         // passa para o próximo passo
});
let tarefas = [{ id: 1, texto: "Estudar JavaScript", feita: false }];
app.get("/tarefas", (req, res) => {
  res.json(tarefas);
});
app.get("/tarefas/:id", (req, res) => {
  const tarefa = tarefas.find(t => t.id === Number(req.params.id));
  if (!tarefa) return res.status(404).json({ erro: "Tarefa não encontrada" });
  res.json(tarefa);
});
app.post("/tarefas", (req, res) => {
  const { texto } = req.body;
  if (!texto) return res.status(400).json({ erro: "Informe o texto" });
  const nova = { id: Date.now(), texto, feita: false };
  tarefas.push(nova);
  res.status(201).json(nova);
});
app.delete("/tarefas/:id", (req, res) => {
  tarefas = tarefas.filter(t => t.id !== Number(req.params.id));
  res.status(204).end();
});
const porta = process.env.PORT ?? 3000;
app.listen(porta, () => console.log(`API em http://localhost:${porta}`));
```

Teste com o navegador (para GET) ou com ferramentas como `curl`, Postman ou Insomnia:

```
curl http://localhost:3000/tarefas
curl -X POST http://localhost:3000/tarefas \
  -H "Content-Type: application/json" \
  -d '{"texto": "Aprender Express"}'
```

Um **middleware** é uma função que fica "no meio do caminho" da requisição: registra logs, valida autenticação, trata erros. Essa ideia é central no Express.

Outras opções de servidor

- **Fastify**: focado em velocidade e validação por esquemas.
- **Hono**: leve e funciona em vários ambientes (Node.js, Bun, serviços de borda).
- **NestJS**: framework completo com TypeScript e arquitetura em módulos, muito usado em empresas.
- **Koa**: criado pela equipe do Express, mais enxuto.

Bancos de dados

Um servidor que guarda dados na memória (como o exemplo acima) perde tudo ao reiniciar. Para persistência, use um banco:

Tipo	Exemplos	Quando usar
Relacional (SQL)	PostgreSQL, MySQL, SQLite	Dados estruturados, relacionamentos, transações. A escolha padrão
Documentos	MongoDB	Dados em formato flexível, parecidos com objetos JSON
Chave-valor / cache	Redis	Cache, filas, sessões

Em vez de escrever SQL à mão, muitos projetos usam um **ORM** ou construtor de consultas, como **Prisma** ou **Drizzle**, que traduz objetos JavaScript em comandos de banco. Com o Prisma, você descreve o modelo em um arquivo de esquema e usa o cliente gerado:

```
model Tarefa {
  id    Int     @id @default(autoincrement())
  texto String
  feita Boolean @default(false)
}
```

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();
await prisma.tarefa.create({ data: { texto: "Estudar banco de dados" } });
const todas = await prisma.tarefa.findMany({ where: { feita: false } });
```

Dica: aprenda o básico de SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `JOIN`). Ele serve para qualquer linguagem e ajuda a entender o que o ORM faz por você.

Variáveis de ambiente e segredos

Configurações que mudam entre computadores e senhas **nunca** ficam no código. Elas vão em variáveis de ambiente, normalmente em um arquivo `.env` que não é enviado ao Git:

```
PORT=3000
DATABASE_URL=postgres://usuario:senha@localhost:5432/meubanco
JWT_SECRET=troque-por-um-segredo-longo
```

Versões recentes do Node.js leem esse arquivo com `node --env-file=.env src/index.js`. Antes disso, usava-se o pacote `dotenv`.

Autenticação e segurança

- **Senhas**: nunca guarde o texto puro. Use um algoritmo de hash próprio para senhas, como **argon2** ou **bcrypt**.
- **Sessões e tokens**: depois do login, o servidor entrega um cookie de sessão ou um token (muito usado: **JWT**). O cliente o envia nas próximas requisições.
- **CORS**: por padrão, o navegador bloqueia chamadas de uma origem para outra. O pacote `cors` libera apenas as origens que você escolher.
- **Validação**: valide todo dado recebido (use o Zod do capítulo 18).
- **Limite de requisições** (rate limiting) e **HTTPS** são obrigatórios em produção.
- Mantenha as dependências atualizadas e siga a lista OWASP Top 10 de falhas comuns.

Tempo real com Socket.IO

Para chats, notificações e painéis ao vivo, o **Socket.IO** mantém uma conexão aberta entre navegador e servidor. No servidor:

```
import { Server } from "socket.io";
const io = new Server(3001, {
  cors: { origin: "http://localhost:5173" }
});
io.on("connection", socket => {
  socket.on("mensagem", texto => {
    io.emit("mensagem", texto);     // reenvia para todos conectados
  });
});
```

No navegador:

```
import { io } from "socket.io-client";
const socket = io("http://localhost:3001");
socket.on("mensagem", texto => console.log("Recebi:", texto));
socket.emit("mensagem", "Olá, pessoal!");
```

Publicando seu projeto

Para colocar um projeto no ar, você precisa de um serviço de hospedagem. Sites estáticos e apps de front-end podem ser publicados em plataformas como Vercel, Netlify ou GitHub Pages; APIs em Node.js rodam em plataformas como Render, Railway, Fly.io ou em um servidor próprio (VPS). Os passos gerais são iguais: enviar o código ao GitHub, conectar o repositório à plataforma, configurar as variáveis de ambiente e definir os comandos de build e de início.

Exercícios

1. Complete a API de tarefas com a rota `PATCH /tarefas/:id` que alterna o campo `feita`.
2. Adicione um middleware que registre o horário e o tempo de resposta de cada requisição.
3. Valide o corpo do `POST /tarefas` com Zod e responda 400 com os erros.
4. Troque o array em memória por um banco SQLite ou PostgreSQL.
5. Conecte o projeto de tarefas em React do capítulo 19 a essa API.
6. Crie rotas de cadastro e login que guardem a senha com hash.

21. Testes automatizados

Por que testar

Testes automatizados são programas que verificam se o seu programa funciona. Eles detectam erros assim que você os introduz, permitem mudar código sem medo de quebrar o que já funcionava e servem como documentação de como as funções devem se comportar.

O ciclo básico de um teste

Todo teste tem três etapas: **preparar** os dados, **executar** a função e **verificar** o resultado esperado. O Node.js já traz um executor de testes nativo:

```
// soma.js
export const somar = (a, b) => a + b;
```

```
// soma.test.js
import test from "node:test";
import assert from "node:assert/strict";
import { somar } from "./soma.js";
test("soma dois números positivos", () => {
  assert.equal(somar(2, 3), 5);
});
test("soma com negativos", () => {
  assert.equal(somar(-1, 1), 0);
});
```

Execute com `node --test`.

Vitest e Jest

Em projetos maiores usa-se uma biblioteca de testes com mais recursos. O **Vitest** (ótimo em projetos com Vite) e o **Jest** têm uma API muito parecida, com `describe`, `it` e `expect`:

```
npm install -D vitest
npx vitest
```

```
import { describe, it, expect, vi } from "vitest";
import { somar } from "./soma.js";
describe("somar", () => {
  it("soma dois números", () => {
    expect(somar(2, 3)).toBe(5);
  });
  it("aceita decimais com aproximação", () => {
    expect(somar(0.1, 0.2)).toBeCloseTo(0.3);
  });
});
describe("outras verificações", () => {
  it("compara objetos e arrays por conteúdo", () => {
    expect({ a: 1 }).toEqual({ a: 1 });
    expect([1, 2, 3]).toContain(2);
  });
  it("verifica erros lançados", () => {
    expect(() => JSON.parse("{")).toThrow();
  });
  it("espia chamadas de função (mock)", () => {
    const callback = vi.fn();
```

```
    [1, 2].forEach(callback);
    expect(callback).toHaveBeenCalledTimes(2);
  });
  it("testa código assíncrono", async () => {
    const valor = await Promise.resolve(42);
    expect(valor).toBe(42);
  });
});
```

Verificações (matchers) mais usadas: `toBe` (igualdade estrita), `toEqual` (igualdade por conteúdo), `toBeTruthy`, `toContain`, `toThrow`, `toHaveBeenCalled`.

Testando interfaces

- **Testing Library** testa componentes (React, Vue e outros) como o usuário os usa: procura botões e textos na tela, clica, digita e verifica o resultado.
- **Playwright** e **Cypress** fazem testes de ponta a ponta (e2e): abrem um navegador de verdade e percorrem o sistema.

```
import { test, expect } from "@playwright/test";
test("a página mostra o título", async ({ page }) => {
  await page.goto("http://localhost:5173");
  await expect(
    page.getByRole("heading", { name: "Minhas tarefas" })
  ).toBeVisible();
});
```

Estratégia: a pirâmide de testes

Tipo	O que testa	Quantidade ideal
Unitário	Uma função isolada	Muitos (rápidos e baratos)
Integração	Várias partes juntas (API e banco)	Alguns
Ponta a ponta (e2e)	O sistema inteiro, como o usuário	Poucos, nos fluxos críticos

TDD

No **desenvolvimento orientado a testes** você escreve o teste antes do código: (1) escreva um teste que falha, (2) escreva o mínimo de código para passar, (3) melhore o código mantendo os testes verdes. Mesmo que você não use TDD sempre, o hábito de pensar nos casos de teste melhora o design das funções.

> **Dica:** teste os casos de borda: lista vazia, zero, números negativos, texto vazio, `null`. É onde os bugs moram.

Exercícios

1. Escreva testes para `ehPar`, `fatorial` e `inverterTexto` (do início do guia), incluindo casos de borda.
2. Use TDD para criar `ehPrimo(n)`: escreva primeiro cinco testes.
3. Escreva um teste para uma função assíncrona que usa `fetch`, substituindo o `fetch` por um mock.
4. Teste as rotas da API de tarefas, chamando `GET /tarefas` e `POST /tarefas`.

22. TypeScript

O que é

O **TypeScript** é o JavaScript com tipos opcionais. Você descreve o formato dos dados e uma ferramenta confere o código **antes** de rodar, apontando erros como passar um texto onde se espera um número. No fim, o TypeScript é convertido em JavaScript comum. É muito usado em projetos profissionais, e frameworks como Angular e NestJS o adotam por padrão.

Primeiros passos

```
npm install -D typescript
npx tsc --init          # cria o tsconfig.json
npx tsc                 # compila os arquivos .ts em .js
```

No `tsconfig.json`, deixe a opção `"strict": true` ligada: ela ativa as verificações mais úteis.

Tipos básicos

```
let nome: string = "Ana";
let idade: number = 25;
let ativo: boolean = true;
let notas: number[] = [8, 9.5, 7];
let par: [string, number] = ["Ana", 25];    // tupla
function somar(a: number, b: number): number {
  return a + b;
}
somar(2, 3);        // ok
// somar("2", 3);   // ERRO de compilação: string não é number
```

Muitas vezes você nem precisa escrever o tipo: o TypeScript **infere** pelo valor (`let x = 5` já é `number`).

Interfaces, types e uniões

```
interface Usuario {
  id: number;
  nome: string;
  email?: string;              // o ponto de interrogação torna opcional
}
type Status = "pendente" | "feito" | "cancelado";   // união de textos literais
function mostrar(u: Usuario, s: Status): string {
  return `${u.nome}: ${s}`;
}
mostrar({ id: 1, nome: "Ana" }, "feito");      // ok
// mostrar({ id: 1, nome: "Ana" }, "talvez");  // ERRO: valor não permitido
```

Estreitamento de tipos

Quando um valor pode ter mais de um tipo, você verifica antes de usar:

```
function tamanho(valor: string | number[]): number {
  if (typeof valor === "string") {
    return valor.length;       // aqui o TypeScript sabe que é string
  }
  return valor.length;         // aqui é array
}
```

Genéricos

Genéricos criam funções e tipos que funcionam com vários tipos mantendo a segurança:

```
function primeiro<T>(lista: T[]): T | undefined {
  return lista[0];
}
const n = primeiro([10, 20]);          // tipo: number | undefined
const s = primeiro(["a", "b"]);        // tipo: string | undefined
```

Boas práticas

- Evite `any`, que desliga a verificação. Se não sabe o tipo, use `unknown` e estreite com verificações.
- Descreva os dados que entram e saem do seu sistema (respostas de API, formulários). O Zod pode gerar tipos TypeScript a partir dos esquemas.
- Migre aos poucos: um projeto JavaScript pode ganhar arquivos `.ts` gradualmente.

> **Dica:** aprenda JavaScript bem antes. TypeScript é uma camada sobre ele, e quem entende a base aprende os tipos em poucos dias.

Exercícios

1. Crie a interface `Produto` (nome, preço, estoque opcional) e uma função que calcule o valor total de uma lista de produtos.
2. Tipifique a função `calcularMedia(notas: number[])`.
3. Crie um tipo `Operacao = "soma" | "subtracao" | "multiplicacao" | "divisao"` e uma calculadora tipada.
4. Converta um arquivo do seu projeto de JavaScript para TypeScript e corrija os erros que aparecerem.

PARTE IV

Prática e referência

Projetos para fixar o conteúdo, um roteiro de estudos de 16 semanas, gabaritos, glossário e links para continuar aprendendo.

23. Projetos para praticar

Por que construir projetos

Ler e resolver exercícios ensina a sintaxe. Construir projetos ensina a **resolver problemas**: dividir uma ideia grande em pedaços, procurar soluções, depurar e terminar o que começou. Escolha projetos que você teria vontade de usar, e entregue versões pequenas antes de acrescentar recursos.

Dez projetos em ordem de dificuldade

Nível	Projeto	O que construir	Conceitos praticados
1	Calculadora	Botões numéricos, operações básicas e botão de limpar	DOM, eventos, funções, condições
1	Jogo de adivinhação	O computador sorteia um número de 1 a 100 e dá dicas de maior ou menor	Math.random, loops, condições
2	Lista de tarefas	Adicionar, concluir, excluir e filtrar tarefas, salvando no navegador	Arrays, objetos, localStorage
2	Cronômetro e timer	Iniciar, pausar e zerar, com contagem regressiva	setInterval, Date, DOM
2	Quiz de perguntas	Perguntas com alternativas, pontuação e tela final	Arrays de objetos, estado
3	Buscador de CEP	Digitar o CEP e preencher o endereço (API ViaCEP)	fetch, async/await, tratamento de erros
3	Previsão do tempo	Buscar uma cidade e mostrar o clima (API Open-Meteo)	APIs, JSON, manipulação do DOM
3	Jogo da velha	Dois jogadores, detecção de vitória e empate, reinício	Matrizes, lógica, funções
4	API de tarefas	Servidor Express com rotas, validação e banco de dados	Node.js, REST, SQL, Zod
4	Chat em tempo real	Salas, apelidos e mensagens ao vivo	Socket.IO, eventos, Express

Detalhando três projetos

Lista de tarefas (nível 2)

Requisitos mínimos: adicionar tarefa, marcar como feita, excluir, mostrar quantas faltam. Evolua com: edição do texto, filtros (todas, pendentes, concluídas), data limite, categorias e ordenação. Depois reescreva em React e, por fim, conecte a uma API sua.

Buscador de CEP (nível 3)

Fluxo: o usuário digita o CEP, o programa remove o hífen, valida o tamanho, chama a API e preenche os campos de rua, bairro e cidade. Trate os casos: CEP inválido, CEP não encontrado e falta de internet. Mostre um indicador de "carregando" enquanto a requisição acontece.

API de tarefas (nível 4)

Rotas para criar, listar, atualizar e remover tarefas, com validação do corpo das requisições, respostas com códigos de status corretos e persistência em banco de dados. Evolua com cadastro de usuários, login, cada usuário vendo só as próprias tarefas e testes automatizados das rotas.

Projeto final: portfólio completo

Reúna o que aprendeu em um sistema que você possa mostrar a recrutadores: por exemplo, um painel financeiro pessoal com cadastro de despesas, categorias, gráficos (Chart.js ou similar), autenticação e uma API própria. Publique o código no GitHub com um `README` que explique o que o projeto faz, como instalar e rodar, as tecnologias usadas e imagens da tela. Publique também a versão online.

Dicas para não travar

- **Divida o problema**: escreva em um papel os passos antes de programar.
- **Comece pelo mínimo**: faça funcionar de forma feia primeiro, depois melhore.
- **Teste a cada passo**: rode o código com frequência, não depois de 100 linhas.
- **Evite o "inferno dos tutoriais"**: depois de assistir a uma aula, refaça o projeto sozinho, sem olhar, e adicione um recurso novo.
- **Faça commits pequenos** e frequentes no Git.
- **Peça ajuda do jeito certo**: descreva o que você esperava, o que aconteceu, a mensagem de erro completa e o menor trecho de código que reproduz o problema.

24. Roteiro de estudos de 16 semanas

Como funciona

O plano supõe de 5 a 6 dias por semana, com 45 a 60 minutos por dia. Em cada semana: três dias de leitura e prática com os exemplos, um dia para os exercícios, um dia para um mini-projeto ou revisão. Se precisar de mais tempo em um tema, estenda a semana: aprender bem é mais importante que cumprir o calendário.

Semana	Tema	Capítulos	Meta da semana
1	Ambiente, variáveis e tipos	1, 2	Rodar código no Node e no navegador; resolver os exercícios de variáveis
2	Operadores e condições	3, 4	Fazer uma calculadora de linha de comando
3	Repetições e funções	5, 6	Resolver FizzBuzz e tabuada; criar 10 funções pequenas
4	Funções avançadas e revisão	6	Entender closures e callbacks; revisar as semanas 1 a 3
5	Arrays	7	Dominar map, filter e reduce em 10 exercícios
6	Objetos, JSON, Map e Set	8	Manipular uma lista de objetos (alunos, produtos)
7	Strings, números e datas	9	Mini-projeto: jogo de adivinhação ou gerador de senhas
8	Erros e módulos	10, 11	Dividir um programa em módulos e tratar erros
9	Classes	12	Modelar uma conta bancária e um carrinho de compras
10	Código assíncrono	13	Consumir uma API pública com async/await
11	DOM e eventos	14	Construir a calculadora e o cronômetro na página
12	Projeto de tarefas, boas práticas e Git	14, 15	Publicar a lista de tarefas no GitHub
13	Node.js e npm	16	Criar um script de linha de comando e um servidor simples
14	Bibliotecas e API	17, 18, 20	Criar uma API com Express e validar com Zod
15	Front-end com framework	19	Recriar a lista de tarefas em React e conectar à API
16	Testes, TypeScript e projeto final	21, 22	Testar a API, publicar o projeto completo

Depois das 16 semanas

- Aprofunde um caminho: **front-end** (React ou Vue, CSS avançado, acessibilidade), **back-end** (bancos de dados, autenticação, arquitetura de APIs, Docker) ou **full-stack** (Next.js ou similar).
- Estude estruturas de dados e algoritmos básicos (pilhas, filas, ordenação, busca). Eles ajudam em entrevistas e no raciocínio.
- Contribua com projetos de código aberto: comece corrigindo erros de digitação na documentação e avance para pequenas correções.

- Mantenha um portfólio com três a cinco projetos bem explicados e o perfil do GitHub organizado.

Apêndice A: Gabarito de exercícios selecionados

Tente resolver sozinho antes de ler. Existem várias soluções corretas: o importante é o seu código funcionar e você entender por quê.

Capítulo 2: Variáveis e tipos

```
console.log(Number("15.5") + 4.5);  // 20
console.log(Number(""));            // 0
console.log(Number(null));          // 0
console.log(Number(undefined));     // NaN
console.log((0.1 + 0.2).toFixed(2)); // "0.30"
```

Capítulos 3 e 4: Operadores e condições

```
// Divisível por 3 e por 5 ao mesmo tempo
const n = 15;
console.log(n % 3 === 0 && n % 5 === 0);   // true
// ?? e || com string vazia
const config = { tema: "" };
console.log(config.tema ?? "claro");   // "" (vazio é um valor válido)
console.log(config.tema || "claro");   // "claro"
// Ano bissexto
function ehBissexto(ano) {
  return (ano % 4 === 0 && ano % 100 !== 0) || ano % 400 === 0;
}
console.log(ehBissexto(2024), ehBissexto(1900), ehBissexto(2000));
// true false true
// Calculadora com switch
function calcular(a, b, operador) {
  switch (operador) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return b === 0 ? "Divisão por zero" : a / b;
    default: return "Operador inválido";
  }
}
```

Capítulo 5: Repetições

```
// FizzBuzz
for (let i = 1; i <= 30; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}
// Contar vogais
function contarVogais(texto) {
  let total = 0;
  for (const letra of texto.toLowerCase()) {
    if ("aeiouáéíóúâêôãõà".includes(letra)) total++;
  }
  return total;
}
console.log(contarVogais("JavaScript é demais")); // 7
```

Capítulo 6: Funções

```
// Média com quantidade variável de notas
const calcularMedia = (...notas) =>
  notas.reduce((soma, n) => soma + n, 0) / notas.length;
console.log(calcularMedia(7, 8, 9));   // 8
// Função que retorna função
const criarSaudacao = saudacao => nome => `${saudacao}, ${nome}!`;
console.log(criarSaudacao("Bom dia")("Ana"));  // "Bom dia, Ana!"
// Fibonacci recursivo
function fibonacci(n) {
  if (n < 2) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log(fibonacci(7));   // 13
```

A versão recursiva do Fibonacci é simples, mas lenta para números grandes, porque repete cálculos. Quando precisar de desempenho, use um loop.

Capítulo 7: Arrays

```
const lista = [3, 8, 1, 9, 4];
console.log(Math.max(...lista), Math.min(...lista));       // 9 1
console.log(lista.reduce((a, n) => a + n, 0));             // 25
// Remover duplicados
const semRepetidos = [...new Set([1, 2, 2, 3, 3, 3])];     // [1, 2, 3]
// Soma dos quadrados dos ímpares de 1 a 10
const total = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  .filter(n => n % 2 !== 0)
  .map(n => n * n)
  .reduce((a, n) => a + n, 0);
console.log(total);   // 165
// Ordenar palavras por tamanho
["banana", "uva", "abacaxi", "kiwi"].sort((a, b) => a.length - b.length);
// ["uva", "kiwi", "banana", "abacaxi"]
```

Capítulo 8: Objetos

```
const alunos = [
  { nome: "Ana", nota: 8 },
  { nome: "Carlos", nota: 5 },
  { nome: "Bia", nota: 9 }
];
// Aluno com a maior nota
const melhorAluno = lista =>
  lista.reduce((melhor, a) => (a.nota > melhor.nota ? a : melhor));
console.log(melhorAluno(alunos));   // { nome: "Bia", nota: 9 }
// Contar letras
function contarLetras(texto) {
  const contagem = {};
  for (const letra of texto) {
    contagem[letra] = (contagem[letra] ?? 0) + 1;
  }
  return contagem;
}
console.log(contarLetras("ola"));   // { o: 1, l: 1, a: 1 }
// Valor total do estoque
const produtos = [
  { nome: "Mouse", preco: 50, quantidade: 4 },
  { nome: "Teclado", preco: 120, quantidade: 2 }
];
const valorEstoque = produtos.reduce((s, p) => s + p.preco * p.quantidade, 0);
console.log(valorEstoque);   // 440
// Contar palavras com Map
const frase = "a vida é bela e a vida é curta";
const contagem = new Map();
for (const palavra of frase.split(" ")) {
  contagem.set(palavra, (contagem.get(palavra) ?? 0) + 1);
}
console.log(contagem.get("vida"));   // 2
```

Capítulo 9: Strings e datas

```
// Palíndromo ignorando maiúsculas e espaços
function ehPalindromo(texto) {
  const limpo = texto.toLowerCase().replaceAll(" ", "");
  return limpo === limpo.split("").reverse().join("");
}
console.log(ehPalindromo("Socorram me subi no onibus em Marrocos")); // true
// Primeira letra de cada palavra em maiúscula
function capitalizar(frase) {
  return frase
    .split(" ")
    .map(p => p.charAt(0).toUpperCase() + p.slice(1).toLowerCase())
    .join(" ");
}
console.log(capitalizar("aprendendo JAVASCRIPT hoje")); // "Aprendendo Javascript Hoje"
// Idade a partir da data de nascimento
function calcularIdade(nascimento) {
  const hoje = new Date();
  let idade = hoje.getFullYear() - nascimento.getFullYear();
  const meses = hoje.getMonth() - nascimento.getMonth();
  const aindaNaoFezAniversario =
    meses < 0 || (meses === 0 && hoje.getDate() < nascimento.getDate());
  if (aindaNaoFezAniversario) idade--;
  return idade;
}
console.log(calcularIdade(new Date(2000, 5, 15)));
```

Capítulos 10 e 13: Erros e código assíncrono

```
// lerJSON
function lerJSON(texto) {
  try {
    return JSON.parse(texto);
  } catch {
    return null;
  }
}
console.log(lerJSON('{"a": 1}'), lerJSON("{ruim"));   // { a: 1 } null
// Contagem regressiva
const esperar = ms => new Promise(resolve => setTimeout(resolve, ms));
async function contagemRegressiva() {
  for (let i = 3; i >= 1; i--) {
    console.log(i);
    await esperar(1000);
  }
  console.log("Já!");
}
contagemRegressiva();
// Tarefas concluídas da API
const resp = await fetch("https://jsonplaceholder.typicode.com/todos");
const todos = await resp.json();
console.log(todos.filter(t => t.completed).length);
```

Capítulo 14: Alternar tema e lembrar a escolha

```
const botao = document.querySelector("#tema");
if (localStorage.getItem("tema") === "escuro") {
  document.body.classList.add("escuro");
}
botao.addEventListener("click", () => {
  const escuro = document.body.classList.toggle("escuro");
  localStorage.setItem("tema", escuro ? "escuro" : "claro");
});
```

Acompanhe com CSS, por exemplo `body.escuro { background: #111; color: #eee; }`.

Apêndice B: Cola rápida de sintaxe

Fundamentos

Conceito	Sintaxe
Variável que muda	let x = 1;
Constante	const PI = 3.14;
Template literal	`Olá, ${nome}!`
Igualdade estrita	a === b
Ternário	cond ? valorSim : valorNao
Valor padrão com ??	x ?? "padrão"
Acesso seguro	obj?.a?.b
Função de seta	const f = (a, b) => a + b;
Parâmetros restantes	function f(...args) {}
Desestruturar array	const [a, b] = lista;
Desestruturar objeto	const { x, y } = obj;
Spread	[...lista, 4] ou { ...obj, z: 1 }
Importar	import { f } from "./arquivo.js";
Exportar	export function f() {}
Classe	class A { constructor() {} }
Tratar erro	try { } catch (e) { }
Async	async function f() { await g(); }

Métodos mais usados de array

Método	Uso
map(fn)	Transforma cada item
filter(fn)	Mantém os itens que passam no teste
reduce(fn, inicial)	Combina tudo em um valor
find(fn) / findIndex(fn)	Primeiro item (ou posição) que atende
some(fn) / every(fn)	Algum / todos atendem
includes(x) / indexOf(x)	Procura um valor
push(x) / pop()	Adiciona / remove no fim
shift() / unshift(x)	Remove / adiciona no início
slice(i, f)	Copia um trecho
splice(i, n, ...novos)	Remove e insere (altera o original)
sort(fn)	Ordena (passe sempre a função de comparação)
join(sep)	Une os itens em texto
flat() / flatMap(fn)	Achata arrays aninhados
Array.from(obj, fn)	Cria array a partir de iterável

Métodos mais usados de string

Método	Uso
length	Quantidade de caracteres
toUpperCase() / toLowerCase()	Muda a caixa
trim()	Remove espaços das pontas
split(sep)	Divide em array
includes(t) / startsWith(t) / endsWith(t)	Verifica trechos
slice(i, f)	Extrai um trecho
replace(a, b) / replaceAll(a, b)	Substitui
padStart(n, c) / padEnd(n, c)	Completa com caracteres
repeat(n)	Repete o texto
localeCompare(outro, "pt-BR")	Compara respeitando acentos

Objetos e utilitários

Recurso	Uso
Object.keys(o) / values(o) / entries(o)	Chaves, valores e pares
Object.fromEntries(pares)	Cria objeto a partir de pares
structuredClone(x)	Cópia profunda
JSON.stringify(x) / JSON.parse(t)	Converte para texto e de volta
Math.round / floor / ceil / trunc	Arredondamentos
Math.max(...lista) / Math.min(...lista)	Maior e menor valor
new Intl.NumberFormat("pt-BR", {...})	Formata números e moeda
Promise.all([...])	Espera várias promessas
setTimeout(fn, ms) / setInterval(fn, ms)	Executa depois / repetidamente

Apêndice C: Glossário

Termo	Significado
API	Conjunto de regras para dois programas conversarem. Pode ser um servidor com endereços (REST) ou as funções de uma biblioteca
Argumento	Valor passado a uma função quando ela é chamada
Array	Lista ordenada de valores, com posições a partir de zero
Assíncrono	Tarefa que termina depois, sem travar o resto do programa
Bundler	Ferramenta que reúne arquivos e pacotes em arquivos otimizados (Vite, webpack)
Callback	Função passada a outra função para ser chamada mais tarde
Closure	Função que mantém acesso às variáveis do escopo onde foi criada
DOM	Representação da página como árvore de objetos que o JavaScript manipula
ECMAScript	Padrão oficial que define a linguagem JavaScript
Escopo	Região do código onde uma variável é visível
Framework	Estrutura que define a organização do projeto e chama o seu código
Hoisting	Comportamento em que declarações são conhecidas antes da linha em que aparecem
JSON	Formato de texto para trocar dados, parecido com objetos JavaScript
Middleware	Função intermediária no caminho de uma requisição
Módulo	Arquivo que exporta funções e valores para outros arquivos
npm	Gerenciador de pacotes do Node.js e repositório de bibliotecas
Objeto	Coleção de pares chave-valor
Parâmetro	Variável que a função declara para receber valores
Promise	Objeto que representa um resultado futuro (sucesso ou falha)
Protótipo	Mecanismo de herança do JavaScript, base das classes
Runtime	Ambiente que executa o JavaScript (navegador, Node.js)
Stack trace	Lista de chamadas que levou a um erro
Tipagem	Regras sobre os tipos de dados; TypeScript adiciona tipos ao JavaScript
Transpilar	Converter código moderno ou TypeScript em JavaScript compatível
XSS	Ataque que injeta scripts em uma página por meio de dados não tratados

Apêndice D: Onde continuar aprendendo

Os endereços abaixo são das fontes oficiais e de materiais de referência amplamente usados. Sites podem mudar de endereço com o tempo; se algum link falhar, pesquise o nome do recurso.

Recurso	Endereço	Para quê
MDN Web Docs	developer.mozilla.org	A principal referência de JavaScript, HTML e CSS (há versão em português)
javascript.info	javascript.info	Tutorial moderno e detalhado, do básico ao avançado
Eloquent JavaScript	eloquentjavascript.net	Livro gratuito online, com exercícios
freeCodeCamp	freecodecamp.org	Cursos gratuitos com prática no navegador
The Odin Project	theodinproject.com	Currículo gratuito de desenvolvimento web, orientado a projetos
roadmap.sh	roadmap.sh	Mapas de carreira para front-end, back-end, Node.js e mais
Node.js	nodejs.org	Download e documentação do Node.js
npm	npmjs.com	Busca de pacotes, versões e downloads
React	react.dev	Documentação e tutorial oficial do React
Vue	vuejs.org	Documentação oficial do Vue
TypeScript	typescriptlang.org	Documentação e playground online
Express	expressjs.com	Documentação do Express
Vitest	vitest.dev	Documentação do Vitest
Can I use	caniuse.com	Compatibilidade de recursos com cada navegador
regex101	regex101.com	Testar e entender expressões regulares
GitHub	github.com	Hospedar código, ler projetos de código aberto e montar portfólio

Uma última orientação

Todo programador, de iniciante a experiente, passa boa parte do tempo lendo erros, pesquisando e testando. Não existe aprender JavaScript "completamente": a linguagem e seu ecossistema evoluem sempre. O que existe é construir uma base sólida, criar o hábito de estudar e saber consultar a documentação. Você já tem tudo isso neste guia. Agora o que falta é abrir o editor e escrever código.

Bons estudos!